RAMBOQUANT

DESIGN GUIDE • v 4436

RamboQuant

Algo Trading Platform

Ramana R Ambore, FRM

Platform Architect & Quantitative Developer

AI-augmented ● built with Claude Code

WEBSITE

ramboq.com

EDITION

July 4, 2026

THE DOCUMENT

About this guide

The RamboQuant **Complete Design Guide** is a top-to-bottom developer + operator onboarding manual for a production algorithmic trading platform. Read cover-to-cover to build a full mental model of the codebase, or jump to any of the 42 sections for focused work. Every subsystem names the exact files; every architectural decision states its trade-off; **Part IX** is a cookbook of change recipes with exact-diff-level guidance.

Generated	Revision	Website
July 4, 2026	v 4436 (f9f019cf)	ramboq.com

Tech Stack

Frontend: SvelteKit • Svelte 5 runes • Vite • ag-grid • @tanstack/svelte-query • Tailwind • Playwright
API: Litestar 2.x • msgspec.Struct • uvicorn • async SQLAlchemy 2.x • asyncpg • httpx • PyJWT (HS256)
Data: PostgreSQL 17 • polars • pandas (broker boundary)
IPC: mmap /dev/shm/ramboq_ticks • UDS /tmp/ramboq_conn.sock • SSE • WebSocket • BroadcastBus
Queues: in-process EventQueue + write_queue • ARQ + Redis worker (separate systemd unit)
Brokers: kiteconnect • dhanhq • growwapi • pyotp (2FA) • KiteTicker WebSocket
Intelligence: Gemini (google-genai) • MCP server • fpdf2 (investor statements) • babel (i18n)
Security: cryptography.Fernet (broker credential encryption) • maxminddb (geo)
Deploy: systemd (api / conn / worker / hook) • nginx • Cloudflare • webhook.ramboq.com auto-deploy

THE AUTHOR

Ramana R Ambore FRM

Platform Architect & Quantitative Developer — RamboQuant LLP AI-AUGMENTED · CLAUDE CODE

30+ years across mainframe modernization and cloud-native financial platforms. Currently **Principal System Analyst** at **Fidelity Investments** — leading billing-platform modernization on AWS + Snowflake and distributed fee-calculation engines — and concurrently the **platform architect + quantitative developer** building **RamboQuant** end-to-end (live at **ramboq.com**), leveraging **Claude Code** as a force multiplier for platform-scale solo output.

FRM (GARP, 2022) and **CFA Level 2** candidate: the derivatives-risk, options-pricing (Black-Scholes / Greeks), and portfolio-analytics theory from those programs materialize directly as RamboQuant's derivatives layer, hedge-proxy β regression, and units-based NAV accounting. The Master's in Computer Science + Fidelity engineering discipline (distributed systems, event-driven architecture, legacy-modernization patterns) keeps a multi-broker, real-time, single-operator platform correct and fast. **NTT Innovation Award** recipient (top-40 global innovator). Based in Merrimack, NH.

Profile: ramanaambore.me

EXPERIENCE & RECOGNITION

CURRENT ROLE	INDUSTRY DEPTH	RECOGNITION
Principal System Analyst	**30+ years FinTech**	**NTT Innovation Award**
Fidelity Investments	Mainframe to cloud-native	Top-40 global innovator
Billing platform modernization on AWS + Snowflake; distributed fee-calculation engines.	Derivatives risk, options pricing (Black-Scholes, Greeks), multi-leg strategy analytics.	Selected globally for innovation contributions in financial-services engineering.

CREDENTIALS

FRM (GARP, 2022) • **CFA Level 2** • **Master's, Computer Science** • Six Sigma Green Belt • IBM Certified DB2 DBA • Sun Certified Java Programmer

Contents

RamboQuant — Complete Design Guide

Part I — Foundation

1. Architecture overview

```
flowchart LR
    Operator((Operator)) -->|browser| FE[SvelteKit frontend<br/>port 5173 / static build]
    FE -->|HTTPS REST| API[Litestar API<br/>port 8502 prod / 8503 dev]
    API -->|asyncpg| DB[(PostgreSQL 17<br/>ramboq / ramboq_dev)]
    API -->|broker SDK| KITE[Kite Connect<br/>+ KiteTicker WebSocket]
    API -->|broker SDK| DHAN[Dhan v2]
    API -->|broker SDK| GROWW[Groww]
    API -->|google-genai| GEMINI[Gemini 2.5 Flash<br/>market + sentiment]
    API -->|smtplib| MAIL[SMTP — Hostinger]
    API -->|requests| TG[Telegram Bot<br/>@RamboQuantBot]
    KITE -->|postback HTTP| API
```

Layer	Tech	Key files
Frontend	SvelteKit + Svelte 5 runes + ag-Grid + hand-rolled SVG charts	`frontend/src/`
API	Litestar 2.x + msgspec.Struct schemas	`backend/api/`
DB	PostgreSQL 17 + SQLAlchemy 2.x async + asyncpg	`backend/api/database.py`, `models.py`
Brokers	Vendor SDKs behind a unified `Broker` ABC	`backend/brokers/`
Background	asyncio tasks spawned at app startup	`backend/api/background.py`

---

2. Tech stack — at a glance

Each choice has a **why/what/how/where** callout inline where relevant (marked by □). Key stacks:

Layer	Tech	Why
API	Litestar 2.x + msgspec	~10× faster JSON encode/decode than pydantic on big payloads
DB	PostgreSQL 17 + SQLAlchemy 2.x async + asyncpg	Fast, reliable, static typing, JSONB for attached_gtts_json blob
Frontend	SvelteKit + Svelte 5 runes + ag-Grid	Smaller bundle, native reactivity, row virtualization for 1000+ ticks/sec
Charts	Hand-rolled SVG (no Chart.js)	150KB saved, tighter control, palette integration
Concurrency	asyncio + single uvicorn worker	Kite token affinity, in-process locks, background task state
Notifications	Telegram (Bot API), SMTP (Hostinger)	Free, reliable, works everywhere
WebSocket	KiteTicker (Twisted reactor □ SSE bridge)	Sub-second LTP updates without burning rate limit

Full rationale for each technology appears as **TECH** callouts throughout this doc (e.g. §3.1 broker abstraction, §4.2 KiteTicker threading, §4.3 background task lifecycle).

---

3. Core architectural principles

3.1 Single source of truth at the broker boundary

The `Broker` abstract base class (`backend/brokers/base.py`) is the **only** place vendor differences should leak. Every route, agent, and background task talks to a `Broker` instance via `get_broker(account)` from the registry.

 TECH — `WHY` Vendor SDKs disagree on EVERYTHING (qty units, status strings, GTT shape). Letting that disagreement propagate past the adapter boundary creates bug surface area in every consumer. `WHAT` The ABC declares ~20 methods (place_order, modify_order, cancel_order, orders, holdings, positions, funds, ltp, quote, historical_data, place_gtt, modify_gtt, cancel_gtt, get_gtts, instruments, profile, holidays, order_status, trades, basket_order_margins). `HOW` New broker? Implement every method, translate to Kite shape in `_normalise_*` helpers, register in `_ADAPTERS`. Capability gaps go in `BrokerCapabilities` — never inline `if broker_id == "groww"`. `WHERE` `backend/brokers/base.py` (ABC); `backend/brokers/adapters/kite.py` / `dhan.py` / `groww.py` (implementations); `backend/brokers/capabilities.py` (matrix).

3.2 Idempotency is the default

Every path that places a broker order or GTT can fire twice — postbacks arrive twice, chase terminals race postbacks, reconcile sweeps re-fire attaches. Four patterns make this safe:

Pattern	Where	What it guards
`attached_gtts_json IS NULL` check	`_fire_template_attach_on_fill`	Double-place TP/SL/Wing at broker
`_TEMPLATE_ATTACH_LOCKS[parer`	Same function	Concurrent races within the same row
`_KILLED_ORDER_IDS` dict with 60-min TTL	`chase.py`	Operator kills landing on stale `broker_order_id`
`MAX(prior, cumulative)` clamp	`_record_partial_fill`	Restart causing cumulative to be added again

When adding a new fill-time side-effect, ask: can my handler fire twice for the same parent? If yes, what's the idempotency check?

3.3 Database is authoritative; in-memory is fast-path

The single uvicorn worker (`--workers 1` in prod — see §4.1) means in-process locks are sufficient, but the DB is still the source of truth. After a restart, every chase loop recovers via `recover_from_db` and re-derives state. Don't store anything operationally meaningful in in-process state without a DB write to back it up.

The `attached_gtts_json` column is a deliberate small-state JSON blob rather than a foreign-key normalized table: - ✓ Atomic write per parent — no half-attached state visible to readers - ✓ Easy to refactor the GTT spec shape (just version the JSON inside) - ✗ Harder to JOIN against; we accept this because GTT inspection is rare

3.4 Async by default, sync when forced

Everything API-facing is `async def` over asyncpg. Broker SDK calls are sync — Kite/Dhan/Groww use `requests` under the hood — so we wrap them in `asyncio.to_thread(...)` to keep the event loop unblocked. The threadpool

sizing is the default (32 workers); we've never seen it saturate because broker API calls are sub-second.

Anti-pattern to avoid: `broker.method()` directly in an `async def` route handler. Even if it returns "fast," a single 2-second hang stalls every other request on that worker.

3.5 Demo mode = signed-out + prod branch

Demo isn't a separate code path — it's a runtime guard at the API boundary (`backend/api/auth_guard.py`) plus a frontend flag pulled from context. The same routes serve authenticated + demo traffic; the guard masks accounts and blocks writes. This means **a feature works in demo the moment it works for read-only sessions** — there's no separate "demo enablement" step to forget.

3.6 Singleton pattern — one instance, everywhere

Several long-lived, expensive-to-construct components are implemented as **process-wide singletons**. The pattern is used deliberately where all of the following are true: (a) construction is expensive (network handshake, mmap open, DB warm), (b) there is exactly one canonical instance per process, (c) many callers need the *same* instance so state stays consistent. Fits nicely with the single-uvicorn-worker guarantee from §4.1 — no cross-worker cache-coherence problem.

Canonical singletons:

Singleton	Module	Purpose
`Connections`	`backend/brokers/connections.py`	Broker session manager (Kite / Dhan / Groww). Owns credentials (decrypted from `broker_accounts` table via Fernet), 2FA, token refresh, IPv6 binding. `connections.Connections()` returns the same instance every time; `rebuild_from_db()` mutates in place.
`TickerManager`	`backend/brokers/kite_ticker.py`	KiteTicker WebSocket wrapper. **One WebSocket per process** — Kite rejects duplicates. Lives inside conn_service when `RAMBOQ_USE_CONN_SERVICE=1`; owns the mmap tick writer at `/dev/shm/ramboq_ticks`.
`MmapTickReader`	`backend/brokers/mmap_ticker.py`	Main-API tick reader. `get_mmap_reader()` returns the module-level `_singleton`. Polls the shared-memory version-word every 50ms; publishes deltas to `BroadcastBus`.
`BroadcastBus`	`backend/api/broadcast.py`	In-process pub/sub. Every SSE client + WebSocket connection subscribes to the same bus for tick + `book_changed` + `position_filled` events.
`GrammarRegistry`	`backend/api/algo/grammar_registry.py`	Agent condition-tree token catalog. Reloaded on `/api/admin/grammar/reload` (mutates the singleton in place; existing agents pick up the new tokens on their next `run_cycle`).
`MarketLifecycle`	`backend/api/algo/market_lifecycle.py`	Per-exchange open / close / close_settled event bus. Polled every 30s by `_task_market_lifecycle`. Handlers registered via `market_lifecycle.register(event, callback)` at import time.

`MetricRegistry` (perf)	`backend/api/persistence/perf_snapshots.py`	Perf-snapshot writer coordinator — buffers metrics until nightly `_task_perf_snapshot` flushes them.

Convention: singleton modules expose a lowercase accessor (`connections.Connections()`, `get_mmap_reader()`, `broadcast_bus()`) rather than a bare module-level global — the accessor lets us swap the implementation in tests (e.g., replace `_singleton` with a fake) and defers construction until first use.

Anti-patterns to avoid:

- **Don't** create local instances of these classes anywhere in route handlers or background tasks. Always go through the accessor. Multiple `Connections()` instances would each mint a fresh Kite session and the second one would evict the first from Kite's session registry.
- **Don't** cache the singleton reference across `rebuild_from_db()` boundaries — the accessor is cheap; long-lived local references miss config changes.
- **Don't** rely on singleton state during startup — `on_startup` order matters. See `backend/api/app.py::on_startup` for the canonical wire-up sequence.

Testing note: pytest fixtures reset `_singleton = None` in `conftest.py::_reset_singletons` between tests so isolation holds. Tests that need a real singleton use the `real_connections` fixture.

---

4. Concurrency model

4.1 Why one uvicorn worker?

`--workers 1` in prod is **intentional** for three reasons:

- **Kite session affinity:** multiple workers would invalidate each other's Kite tokens because Kite enforces one active session per IP.
- **In-process locking is enough:** all locks are `asyncio.Lock` instances; we never need multiprocess coordination.
- **Background tasks need shared state:** the trail-stop poller's in-memory state (`_TEMPLATE_ATTACH_LOCKS`, the ticker manager's `_tick_map`) is process-scoped. Multi-worker would require Redis or similar.

If we ever scale horizontally we'd need to externalize: tokens → DB, locks → DB advisory locks or Redis, ticker state → a separate fanout service.

4.2 KiteTicker threading

`KiteTicker` runs Twisted internally — **all WebSocket callbacks fire on a Twisted reactor thread**, not the asyncio event loop. The `TickerManager` bridges this: - Twisted thread writes `_tick_map[token] = ltp` under a `threading.Lock` - Async handlers read via `get_ltp(token)` — same lock, briefly held

The lock is non-reentrant and the critical section is O(1) so no deadlock risk.

 TECH — `WHY` Twisted reactors can't see asyncio's event loop, so we can't `await` from a tick callback. Lock-protected dict is the simplest viable bridge. `WHAT` `TickerManager._tick_map: dict[int, float]` + `_tick_lock: threading.Lock`. `HOW` Tick handler does `with self._tick_lock: self._tick_map[token] = ltp`. Async reader does the same under the lock, briefly. `WHERE` `backend/brokers/kite_ticker.py`.

If you add anything that runs on the Twisted side, never call `asyncio.run_coroutine_threadsafe` without testing both directions of the round-trip. The reactor doesn't know about asyncio's event loop.

4.3 Background task lifecycle

All background tasks are spawned in `app.on_startup` via `asyncio.create_task(...)`. They run forever; cancellation only happens at app shutdown. Each task is responsible for its own error handling — an uncaught exception kills the task silently. Every task body should be:

```
async def _task_X():
    while True:
        try:
            ...real work...
        except Exception as e:
            logger.exception(f"_task_X iteration failed: {e}")
        await asyncio.sleep(interval)
```

The `try/except` around the loop body is non-negotiable. We've burned hours debugging "why did the trail stop go silent" only to discover an unhandled `KeyError` ate the task three days earlier.

4.5. Data layer — implementation detail

This section is the "if you're modifying the data layer, here's the actual shape" reference. The data layer = SQLAlchemy 2.x async ORM + PostgreSQL + asyncpg driver. Read this end-to-end before changing any table.

4.5.1 Topology

```
flowchart TD
    subgraph appstart [App startup]
        START[backend/api/app.py on_startup]
        START --> INIT[init_db]
    end

    subgraph dblayer [Data layer]
        INIT --> META[Base.metadata.create_all]
        INIT --> ALT[ALTER TABLE ... IF NOT EXISTS<br/>idempotent migrations]
        INIT --> SEED[seed_grammar_tokens · seed_agent_templates<br/>seed_agents · seed_settings ·
seed_templates<br/>seed_global_pinned · seed_hedge_proxies]
    end

    subgraph runtime [Runtime — every request]
        REQ[Route handler] -->|async with| AS[async_session]
        AS -->|asyncpg pool| PG[(PostgreSQL 17)]
        AS -->|expire_on_commit=False| RD[ORM rows readable post-commit]
    end

    INIT -.startup-only.-> META
    META -.then.-> ALT
    ALT -.then.-> SEED
    SEED --> READY[App ready]
    READY --> REQ
```

4.5.2 File map

File	Purpose
`backend/api/database.py`	Engine + session factory + `init_db` (the only place we touch DDL)
`backend/api/models.py`	Every SQLAlchemy declarative model. One file by convention so the data shape is one grep away.
`backend/api/schemas.py`	msgspec.Struct wire types. Mirror of models for HTTP responses.
`backend/shared/helpers/settings`	`SEEDS` list + cached settings reader (`get_int / get_float / get_bool / get_string`)
`backend/api/algo/templates_seed`	`SYSTEM_TEMPLATES` + the seeder
`backend/api/algo/grammar.py`	`_SYSTEM_TOKENS` + grammar registry seeder
`backend/api/cache.py`	In-memory TTL cache with per-key locking (NOT a substitute for the DB; cache invalidates on PATCH)

4.5.3 Engine + session factory

`database.py` is small and worth reading in full. Key shape:

```
# backend/api/database.py
engine = create_async_engine(
    DATABASE_URL,            # postgresql+asyncpg://...
    echo=False,
    pool_size=5,             # max connections kept warm in the pool
    max_overflow=10,         # extra one-off connections when pool exhausted
)

async_session = async_sessionmaker(
    engine,
    expire_on_commit=False,  # load-bearing — see 4.5.5
    class_=AsyncSession,
)

async def init_db() -> None:
    """Idempotent: safe to run on every startup."""
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
        # ALTER TABLE ... IF NOT EXISTS for every column added after
        # the table's initial creation. We don't use Alembic.
        await conn.execute(text(
            "ALTER TABLE algo_orders ADD COLUMN IF NOT EXISTS filled_quantity INTEGER"
        ))
        # ... many more such ALTERs ...
    # Seeders run after the DDL block — see §4.5.7
```

 TECH — Why `expire_on_commit=False` — `WHY` After `commit()`, SQLAlchemy's default is to expire all ORM attributes on the committed rows, so the next attribute access triggers a fresh SELECT. That's catastrophic in our codebase because several handler paths commit then immediately read attributes (chase reconcile attach queue, retry_template, postback fallback match). With expire-on-commit we'd issue redundant SELECTs per commit. `WHAT` Setting this to `False` keeps the in-Python row state intact after commit. `HOW` Set globally on the `async_sessionmaker`. Never override per-session — consistency matters. `WHERE backend/api/database.py::async_session`.

4.5.4 Models — how to add / modify

`backend/api/models.py` is the canonical schema. Every table is a `Mapped[]`-typed class:

```python
# backend/api/models.py
class AlgoOrder(Base):
    __tablename__ = "algo_orders"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    broker_order_id: Mapped[str | None] = mapped_column(String(64), nullable=True, index=True)
    account: Mapped[str] = mapped_column(String(32), index=True)
    symbol: Mapped[str] = mapped_column(String(64), index=True)
    quantity: Mapped[int] = mapped_column(Integer)
    filled_quantity: Mapped[int | None] = mapped_column(Integer, nullable=True)
    status: Mapped[str] = mapped_column(String(32), default="OPEN", index=True)
    mode: Mapped[str] = mapped_column(String(8), default="live", index=True)
    template_id: Mapped[int | None] = mapped_column(
        ForeignKey("order_templates.id", ondelete="SET NULL"), nullable=True
    )
    attached_gtts_json: Mapped[str | None] = mapped_column(Text, nullable=True)
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), index=True
    )
    # ... ~30 more fields, see actual file ...

    __table_args__ = (
        Index("ix_algo_orders_mode_status", "mode", "status"),  # composite, Sprint E
    )
```

Rules when adding a column: 1. Add the `Mapped[]` declaration to the model class. 2. Add an idempotent `ALTER TABLE algo_orders ADD COLUMN IF NOT EXISTS ...` to `init_db`. **Never write an Alembic migration** — our pattern is idempotent ALTERs at startup. 3. New column should be `nullable=True` with sensible default unless you're guaranteed to backfill all rows. 4. Composite indexes go in `__table_args__` and need an `ALTER` to ensure existence on rebuild. The convention: `Index("ix_<table>_<cols>", "col1", "col2")`.

Rules when adding a table: 1. Subclass `Base`, set `__tablename__`. 2. Declarative create happens automatically via `Base.metadata.create_all`. 3. Add seeded rows via a new seeder function called from `init_db`. 4. Add the corresponding msgspec.Struct in `schemas.py` if the table is operator-visible.

4.5.5 Session lifecycle in handlers

The canonical handler pattern:

```python
@get("/example", guards=[auth_or_demo_guard])
async def example(self, request: Request) -> ExampleResponse:
    async with async_session() as s:
        # Reads + writes happen here
        rows = (await s.execute(
            select(AlgoOrder).where(AlgoOrder.status == "OPEN")
        )).scalars().all()
        # Mutate
        for r in rows:
            r.last_seen = datetime.now(timezone.utc)
        # Single commit at the end of a logical group
        await s.commit()
    return ExampleResponse(rows=[_to_info(r) for r in rows])
```

Anti-patterns to avoid:

- ❌ Holding a session across `await` to a broker SDK call. The broker call could take 5 seconds; the session holds a connection from the pool the whole time. Wrap the broker call in `asyncio.to_thread` OR exit the `async with` block first and re-open after.
- ❌ Committing inside a loop without batching. Each commit is a round-trip — if you have 100 rows to update, batch into one commit at the end.
- ❌ `select(AlgoOrder)` without a WHERE clause and no LIMIT. Full-table scans land in production logs eventually; always paginate operator-visible queries.
- ❌ Mutating an ORM row from one session and reading from another within the same request. Use one session per logical unit of work.

4.5.6 Idempotent migrations pattern

We don't use Alembic. Every schema change is an `ALTER TABLE ... IF NOT EXISTS` in `init_db`:

```
async def init_db():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
        await conn.execute(text(
            "ALTER TABLE algo_orders ADD COLUMN IF NOT EXISTS filled_quantity INTEGER"
        ))
        await conn.execute(text(
            "ALTER TABLE algo_orders ADD COLUMN IF NOT EXISTS sl_trail_pct NUMERIC(8, 4)"
        ))
        # ... dozens more ...
```

This pattern was chosen because: - ✅ Zero ops overhead — no migrations folder, no version cursor, no rollback worry. - ✅ Deploy is just `git pull + restart`. The new column appears the moment the restart finishes. - ✅ Branch-switching works — if dev is ahead of main, switching back to main doesn't break (the column exists; the code that uses it is gone). - ✅ No DOWN migrations. We don't drop columns; if we want to remove a field, we stop reading from it but leave the column in place. - ⚠ Cannot rename columns easily. The workaround: add a new column, dual-write for one deploy, switch reads, then stop writing the old.

The pattern is suitable for small-team prod with infrequent destructive changes. If we ever need ten-engineer concurrent migrations, this stops scaling and we'd move to Alembic.

4.5.7 The seeders — bootstrapping defaults

Seven seeders run at startup. Six fire from inside `init_db()` (in `backend/api/database.py`); `seed_hedge_proxies` is registered as its own `on_startup` coroutine in `app.py` so it runs after the DDL block:

Seeder	Where it lives	What it seeds	Operator-overridable?
`seed_grammar_tokens`	`backend/api/algo/grammar.py`	Grammar tokens (metric/scope/op/channel/format/tem	Toggleable `is_active`; cannot delete system tokens
`seed_agent_templates`	`backend/api/algo/template_regis`	Reusable agent templates referenced by built-in agents	Toggleable; refresh on boot
`seed_agents`	`backend/api/algo/agent_engine.p`	10 built-in agents (6 loss-, *3 expiry-*, 1 manual)	Editable; seeder force-resets `schedule` field + force-inactives built-in summary agents on every boot

Seeder	Where it lives	What it seeds	Operator-overridable?
`seed_settings`	`backend/shared/helpers/settings`	Settings rows (`alerts.cooldown_minutes`, `chase.max_consecutive_errors`, etc.)	Yes — operator edits via `/admin/settings`; seeder preserves `value` and only refreshes metadata
`seed_templates`	`backend/api/algo/templates_seed`	4-default order templates (`default-bull`, `default-long-option`, `default-bear`, `default-short-vol`)	Partial — system templates are overwritten on every restart, but operator's saved copies (different `user_id`) survive
`seed_global_pinned`	`backend/api/routes/watchlist.py`	Pinned global watchlists (Markets, Default)	Editable per-user; global rows refresh
`seed_hedge_proxies`	`backend/api/routes/hedge_proxie` (runs from `app.on_startup`, not `init_db`)	Six default pairs (GOLDBEES GOLD/GOLDM, etc.)	Editable via `/admin/settings → Hedge proxies`

The recurring tension in seeders: **how much to overwrite on every boot vs preserve operator changes?** The pattern: - **Description / schema / metadata** — always refreshed (code is the source of truth) - **`value` / `conditions` / `actions` / `events`** — preserved on existing rows (operator's overrides) - **`status`** — preserved EXCEPT for built-in summary agents which force-reset to `inactive` (see `seed_agents`) - **New rows added** with whatever default the SEEDS const ships

If you're adding a new seeder, follow this pattern. The audit-friendly path is `INSERT ... ON CONFLICT DO UPDATE SET <metadata only>` so existing values can't be clobbered.

4.5.8 Settings cache layer

`backend/shared/helpers/settings.py` exposes `get_int`, `get_float`, `get_bool`, `get_string` readers backed by an in-process dict cache:

```
_SETTINGS_CACHE: dict[str, Any] = {}
_CACHE_GENERATION = 0

def get_int(key: str, default: int) -> int:
    val = _SETTINGS_CACHE.get(key)
    if val is None:
        return default
    return int(val)
```

The cache loads on first read + invalidates on every PATCH (the route handler bumps `_CACHE_GENERATION` and clears `_SETTINGS_CACHE`). This means: - ✅ Hot-path reads are O(1) dict lookup — settings are checked thousands of times per minute on background tasks. - ✅ Operator edits take effect on the next read, no service restart. - ⚠ Multi-worker would need per-worker invalidation (we're single-worker — see §4.1 — so this isn't an issue).

When adding code that reads a setting, **always go through `get_*` helpers**. Never `SELECT ... FROM settings WHERE key = ...` in a hot path.

4.5.9 `attached_gtts_json` — the small-state JSON blob pattern

`AlgoOrder.attached_gtts_json` deserves its own subsection because it's the load-bearing column for template attach (§9):

```
# Persisted shape (string-encoded JSON)
[
    {
        "kind": "gtt",
        "label": "TP",
        "id": "abc123",                    # broker GTT id
        "current_trigger": 250.0,
        "sl_trail_pct": null,
        "tp_trigger": 250.0,
        "highest_ltp": 200.0,              # set if sl_trail_pct is non-null
        "lowest_ltp": 200.0,
        "sibling_id": null                 # set for Groww emulated OCO pairs
    },
    {
        "kind": "wing",
        "label": "Wing BUY",
        "id": "def456",                    # broker order id (not GTT — wings are real positions)
        "qty": 50,
        "symbol": "NIFTY24APR21500PE"
    },
    ...
]
```

Why a blob and not a child table? Three reasons: - Atomic write per parent — readers never see "half-attached" state. - Easy to refactor the spec shape (just version the JSON inside, no migration). - GTT inspection is rare — we don't JOIN against it. When we read it, we read the whole bracket anyway.

Idempotency rule: the `_fire_template_attach_on_fill` function checks `attached_gtts_json IS NULL` before writing. Once populated, it's never re-attached (see §9 for the full guard chain). To force a re-attach, the operator hits `POST /orders/algo/<id>/retry-template`.

4.5.10 Pool sizing + connection accounting

Defaults: - `pool_size=5` connections kept warm - `max_overflow=10` extra one-off connections under burst - `pool_pre_ping=True` checks the connection is alive before checkout

We've never seen pool exhaustion in prod because: - Single worker (§4.1) caps concurrent request handlers at the asyncio scheduler's natural limit. - Background tasks use SHORT sessions (`async with` inside the loop body, not around the loop).

Symptoms of pool exhaustion (if you ever see them): - Slow request handlers despite low DB CPU. - `TimeoutError: QueuePool limit of size 20 overflow 10 reached` in logs.

Fix path: investigate which handler is holding sessions across `await` to a slow external call. **Don't bump `pool_size` first** — it's almost always a code-shape problem, not a sizing one.

4.5.11 Demo masking — at the data layer boundary

Read paths for demo sessions mask account values via `mask_column` (§22). The masking happens **at the route layer**, not at the data layer:

```
# Wrong — masking inside the ORM
class AlgoOrder(Base):
    account: Mapped[str] = mapped_column(String(16))
```

```
    @property
    def account_display(self):
        return mask_column(self.account)

# Right — masking at the route handler
@get("/orders")
async def list_orders(self, request: Request) -> list[OrderInfo]:
    rows = await self._fetch()
    is_demo = request.state.is_demo
    return [
        OrderInfo(account=mask_column(r.account) if is_demo else r.account, ...)
        for r in rows
    ]
```

This keeps the data layer free of presentation concerns and avoids subtle bugs (e.g. ORM expressions seeing masked values during a JOIN).

4.6 Database schema overview

RamboQuant's data model spans 35+ SQLAlchemy tables, split into logical domains. Two PostgreSQL databases: `ramboq` (prod, main branch) and `ramboq_dev` (dev branches). All tables live in the branch-local DB except `broker_accounts`, which is shared.

Table categories

Auth + User Management — User identity, email verification, roles, compliance:

Table	Purpose	Key columns
`users`	Operator profiles + LP investor records. Unified table for internal staff + external LPs.	id (PK), account_id (unique), username, role (designated/trader/risk/admin/partner), email, pan, kyc_verified, contribution_date, share_pct, assigned_accounts, assigned_strategies, token_version (for force-logout)
`auth_tokens`	One-time email verification + password-reset tokens. Single table with `purpose` discriminator.	id (PK), user_id (FK), purpose (verify|reset), token (unique), expires_at, used_at
`impersonation_events`	Audit of admin/designated impersonating a partner. Tracks session start + end.	id (PK), actor_username, target_username, started_at, ended_at, end_reason
`investor_tokens`	Long-lived URL tokens for LP-facing portal access. Token IS the credential.	id (PK), user_id (FK), token (unique), expires_at, revoked_at, last_visit_at, visit_count
`investor_events`	LP capital ledger — subscriptions, redemptions, bootstrap events. Source of truth for units-based NAV math.	id (PK), user_id (FK), event_type (subscription|redemption|bootstrap), event_date, amount, nav_per_unit, units_delta
`monthly_statements`	Audit of auto-emailed LP statements. One row per (user, period_year, period_month).	id (PK), user_id (FK), period_year, period_month, generated_at, sent_at, recipients_json, pdf_size_bytes, error

Broker Connection — Account credentials + market metadata:

Table	Purpose	Key columns
`broker_accounts`	Shared table (ramboq DB only) storing encrypted broker credentials for all branches.	id (PK), account (unique, e.g. ZG0790), broker_id (kite|dhan|groww), user_id (FK), api_key_enc, access_token_enc, source_ip, priority, poll_priority, circuit_breaker_enabled, display_order
`market_holidays`	Exchange holiday calendar (NSE/MCX/CDS). Seeded from broker API, cached.	id (PK), exchange, holiday_date (unique per exchange)
`market_special_sessions`	Special trading sessions (e.g. Muhurat trading). Operator-editable overrides.	id (PK), exchange, date, start_time, end_time, reason

Watchlists — User-defined symbol groups:

Table	Purpose	Key columns
`watchlists`	Named list containers. Global rows (is_global=True, user_id=NULL) shared across all users.	id (PK), user_id (FK nullable), name, is_global, is_default, is_pinned, sort_order
`watchlist_items`	Individual symbols in a watchlist. Includes operator-supplied alias.	id (PK), watchlist_id (FK), tradingsymbol, exchange, alias (optional label), sort_order

Orders + Execution — Core order lifecycle and attribution:

Table	Purpose	Key columns
`algo_orders`	Master order row. Spans manual + agent-fired + template-attached + chase iterations. Mode: sim/paper/live/replay/shadow.	id (PK), account, symbol, exchange, transaction_type (BUY/SELL), quantity, filled_quantity (cumulative across partials), initial_price, current_limit (re-quoted during chase), fill_price, status (OPEN/FILLED/REJECTED/CANCELLED), mode, engine (manual/sim/paper/live/replay/shadow/expiry), agent_id (FK nullable), broker_order_id, template_id (FK), attached_gtts_json (JSON list of {kind, label, id, …}), basket_tag, parent_order_id (for TP/SL children), strategy_id (FK), request_id (for audit drill-through)
`algo_order_events`	Append-only timeline per order. One row per state transition (placed, chase_modify, fill, unfill, reject, cancel).	id (PK), order_id (FK), ts, kind (placed|chase_modify|fill|unfill|reject|cancel|postback|…), message, payload_json
`algo_events`	Legacy event log (pre-AgentEvent era). Deprecated; kept for compatibility.	id (PK), algo_order_id (FK nullable), event_type, detail, timestamp
`strategies`	Named bucket for order attribution. Owns lots + provides per-strategy P&L rollup.	id (PK), slug (unique), name, description, owner_user_id (FK nullable), capacity_cap_inr, target_volatility, is_active
`strategy_lots`	Per-strategy FIFO lot ledger. Opens on fill, closes when counter-direction consumes it. Authoritative for per-strategy P&L.	id (PK), strategy_id (FK), open_order_id (FK nullable), account, symbol, exchange, side (B/S), qty, remaining_qty, open_price, close_price, realized_pnl, opened_at, closed_at

Agents + Conditions — Rule engine and alert infrastructure:

Table	Purpose	Key columns
`agents`	Declarative rules: condition tree alert actions. Includes loss alerts, expiry alerts, user-defined custom agents.	id (PK), slug (unique), name, long_name (3-part descriptor), description, conditions (JSONB tree), events (alert channels), actions (order/cancel/close side-effects), scope (per_account|all_accounts), schedule (market_hours|continuous|…), cooldown_minutes, fire_at_time (gate to HH:MM window), trade_mode (paper|live per-agent override), status (active|inactive|cooldown|completed), lifespan_type (persistent|one_shot|n_fires|until_date), tier (critical|high|medium|low for noise reduction), topic (agent grouping tag), digest_window_sec (buffer outgoing alerts), debounce_minutes, condition_first_true_at, tags (JSONB list), blackout_windows (quiet hours)
`agent_events`	Alert events fired by agents. Persisted for operator inspection + MCP queries.	id (PK), agent_id (FK), event_type (fired|suppressed|error|…), trigger_condition (the condition leaf that fired), detail, sim_mode (simulator flag), timestamp
`grammar_tokens`	Token catalog (metrics, scopes, operators, channels, actions). Extensible alphabet for condition/alert/action trees.	id (PK), grammar_kind (condition|notify|action), token_kind, token (name), value_type, resolver (dotted path to function), params_schema, enum_values (JSONB), is_active, is_system

NAV + Performance — Investor slicing and daily metrics:

Table	Purpose	Key columns
`nav_daily`	Daily firm-level NAV snapshot. Written after broker positions settle. Authoritative for all investor slicing.	id (PK), as_of_date (unique), nav, cash_total, positions_mtm, holdings_mtm, accounts_snapshot (JSONB list), note
`strategy_snapshots`	Daily per-strategy P&L rollup. Charts the strategy performance curve.	id (PK), strategy_id (FK), as_of_date, open_lots_count, open_notional, realised_pnl, unrealised_pnl, margin_allocated
`perf_snapshots`	Nightly static + runtime metrics per page. Used for trend analysis + performance budgets.	id (PK), page_or_route, metrics_json (JSONB), static_metrics_json, captured_at

Data Snapshots — Intraday market state + persistent cache:

Table	Purpose	Key columns
`daily_book`	Intraday snapshot of positions, holdings, or funds per account per symbol. Captured at market close, useful when markets closed.	id (PK), kind (positions|holdings|funds), account, symbol, exchange, qty, avg_price, ltp, pnl, pnl_pct, date, captured_at, segment (NSE|MCX|…)
`ohlcv_daily`	5-year OHLCV history. Persistence tier 2 fallback for chart data.	symbol, exchange (PK), date (PK), open, high, low, close, volume
`instruments_snapshot`	Per-exchange symbol token map. Refreshed daily.	id (PK), exchange, date, payload (JSONB full instruments dict), row_count

Table	Purpose	Key columns
`holidays_snapshot`	Exchange holiday sets per year. Immutable once year closes.	id (PK), exchange, year, dates_json (JSONB list)
`intraday_bars`	5/15/30/60-minute bars. 90-day rolling retention.	id (PK), symbol, exchange, date, interval (5min|15min|30min|60min), bar_ts, open, high, low, close, volume
`movers_snapshots`	Nightly snapshot of top movers (NIFTY, NIFTYNXT50, etc). Pre-computed so /pulse doesn't timeout.	id (PK), index_symbol, snapshop_date, movers_json (JSONB list)

Audit + Compliance — Forensic trails:

Table	Purpose	Key columns
`audit_log`	HTTP request + mutation audit trail. Every write captured by middleware + explicit handlers.	id (PK), actor_user_id, username, role, action, category (order.place|order.fill|agent.action|system.nav|…), method, path, target_type, target_id, status_code, summary, request_id (FK to al go_orders for drill-through), client_ip, created_at
`mcp_audit`	Mutations initiated via MCP (Claude Code research mode). Tracks tool calls + results.	id (PK), user_id (FK), tool_name, args_redacted, result_status, result_summary, request_id, created_at
`admin_email_events`	Audit of admin-triggered alert sends (e.g. manual test notifications).	id (PK), admin_user_id, recipient_email, subject, body_preview, sent_at, error
`visitor_log`	Minimal analytics — timestamps of /auth/login + visitor count.	id (PK), visitor_ip, last_seen_at, visitor_count

Market Lifecycle + Background Jobs — System state:

Table	Purpose	Key columns
`market_lifecycle_events`	Audit log of market open/close transitions per exchange. Indexed for operator drill-down.	id (PK), exchange, event_type (nse:open|nse:close|nse:close_settled|…), fired_at, captured_at
`code_metrics_snapshots`	Captured per release (commit SHA). Query count, test counts, response times.	id (PK), release_version, static_metrics_json, runtime_metrics_json, captured_at

Configuration + Extensibility:

Table	Purpose	Key columns
`settings`	DB-backed tunables: alert cooldown, retry counts, market hours, etc. Operator-editable via `/admin/settings`.	id (PK), category, key (unique), value_type (int|float|bool|string), value (operator-editable), default_value, description, schema (JSON validation), units
`order_templates`	Reusable bracket recipes. System templates (default-bull, default-bear, etc) seeded at boot; operator saves custom copies.	id (PK), user_id (FK nullable), name, slug, owner_user_id (FK nullable), tp_pct, sl_pct, wing_premium_pct, wing_strike_offset, wing_qty, tp_scales_json, tp_order_type, sl_trail_pct, is_system

Table	Purpose	Key columns
`hedge_proxies`	Cross-reference between holdings (GOLDBEES) and option roots they hedge (GOLD). Includes β regression.	id (PK), proxy_symbol, target_root, is_active, note, beta, correlation, regression_at
`research_threads`	Persistent Chat threads in `/admin/research` (MCP Lab). One row per thread; messages stored as JSON.	id (PK), created_by_user_id (FK), title, slug, messages_json (JSONB), summary, created_at, updated_at
`sim_recordings`	Deterministic event logs for replay. Captures every state mutation so operator can re-run identical scenarios.	id (PK), label, scenario, seed_mode, started_at, ended_at, duration_sec, tick_count, event_count, payload (JSONB event stream), owner_user_id (FK)
`sim_iterations`	Multi-run coordinator. First iteration references itself; others reference iteration 1 via `parent_run_id`.	id (PK), slug (unique), parent_run_id (FK self-ref nullable), iteration_index, iterations_total, regime, seed, started_at, ended_at, end_reason, params_json, summary_json

News + Market Reports:

Table	Purpose	Key columns
`market_report`	Single-row cache (id=1) for daily market summary from Gemini API.	id (PK, always 1), content, cycle_date, refreshed_at, generated_at
`news_headlines`	RSS headlines cache. Pre-filtered on /market page.	id (PK), link (unique), title, summary, published_at, source, category

---

4.7 Table relationships

Simplified ERD (ASCII, readable in PDF):

```
users ──────────┬── broker_accounts (shared DB)
                ├── algo_orders ─── algo_order_events
                │         ├─ agent_id → agents
                │         ├─ strategy_id → strategies ─── strategy_lots
                │         ├─ template_id → order_templates
                │         └─ parent_order_id → algo_orders (self-ref TP/SL)
                ├── agent_events ← agents
                ├── watchlists ─── watchlist_items
                ├── investor_events (capital ledger)
                ├── investor_tokens (URL credentials)
                ├── monthly_statements
                ├── sim_recordings
                ├── research_threads
                └── auth_tokens

nav_daily ──────────── (master NAV — no FK, firm aggregate)
strategy_snapshots — strategies
daily_book ─────────── (account/symbol/kind snapshot, no FK)
perf_snapshots ──────── (page metrics, no FK)
```

```
market_lifecycle_events, code_metrics_snapshots, settings,
order_templates, hedge_proxies, grammar_tokens ——— (configuration, mostly no FK)

Persistence:
  ohlcv_daily, instruments_snapshot, holidays_snapshot,
  intraday_bars, movers_snapshots ——— (market data cache, no FK)

Audit:
  audit_log — request_id → algo_orders (drill-through)
  mcp_audit — user_id → users
  admin_email_events — admin_user_id → users
```

Key foreign-key patterns: - `algo_orders.agent_id` → agents.id: `ON DELETE SET NULL` (preserve order history if agent deleted) - `agent_events.agent_id` → agents.id: `ON DELETE CASCADE` (retire agent history when agent deleted) - `algo_orders.parent_order_id` → algo_orders.id: `ON DELETE SET NULL` (self-ref for TP/SL children) - `algo_order_events.order_id` → algo_orders.id: `ON DELETE CASCADE` (timeline only meaningful with parent) - `investor_events.user_id` → users.id: `ON DELETE RESTRICT` (capital ledger is audit trail; explicit operator action required to delete LP) - User.id: multiple targets use `ON DELETE CASCADE` (auth_tokens, investor_tokens, monthly_statements) or `SET NULL` (impersonation, strategy owner)

4.8 Retention policies

Data retention is staggered — critical audit trails kept longer than ephemeral cache. Configured via `settings` table; operator can adjust retention via `/admin/settings` → Retention.

Table	Retention	Config key	Rationale
`ohlcv_daily`	5 years	(hardcoded)	SEBI Cat-III backtest horizon
`instruments_snapshot`	7 days	(hardcoded)	Symbol token changes rarely; old snapshots are cheap to drop
`holidays_snapshot`	Forever	(hardcoded)	Reference data — rarely used but occasionally queried for audits
`intraday_bars`	90 days	(hardcoded)	Intraday charts old after 3 months; 90-day window covers quarterly strategy review
`algo_events`	30 days	`retention.algo_events_days`	Operational log; older entries rarely queried
`algo_order_events`	90 days	`retention.algo_order_events_da`	Order timeline — longer window for compliance disputes
`auth_tokens`	7 days after expiry	`retention.auth_tokens_days`	One-time verify/reset tokens; ephemeral by design
`mcp_audit`	90 days	`mcp.audit_retention_days`	MCP tool calls (research mode) — compliance audit trail
`audit_log`	365 days	`retention.audit_log_days`	User action audit trail — 1-year window for disputes
`nav_daily`	Forever	(hardcoded)	Investor reporting — Cat-III requires 8-year retention

Table	Retention	Config key	Rationale
`daily_book`	Forever	(hardcoded)	Intraday snapshots become the P&L source after market close
`investor_events`	Forever	(hardcoded)	Capital ledger — units-based NAV depends on full history
`monthly_statements`	Forever	(hardcoded)	Investor statements are permanent records
`strategy_snapshots`	Forever	(hardcoded)	Strategy performance is historical record
All others	Forever	—	Configuration, metadata, non-critical operational logs kept for full history

Cleanup mechanism: nightly cron (`03:10 IST` and staggered) runs `DELETE FROM <table> WHERE created_at < now() - interval`. Per-table cleanup is idempotent — multiple runs don't corrupt state.

Operator override: Set retention to `0` in `/admin/settings` to disable deletion for that table (useful during active investigation).

4.9 Metrics + Performance tracking

RamboQuant tracks two orthogonal signals: **static code health** (per-file metrics) and **runtime Web Vitals** (per-page response times, JavaScript heap pressure). Both are persisted nightly to `perf_snapshots` and visualized in `/admin/perf`.

 TECH — Why centralized metrics — `WHY` Operator feedback (Jun 2026): "dropdown lag", "refresh button stuck", "frontend heap keeps growing". A single dashboard surfacing cyclomatic complexity + LCP + JS heap reveals systemic issues (subscription leaks, long tasks, state bloat) before they degrade UX. `WHAT` Nightly snapshot of static + runtime metrics, persisted with ISO8601 timestamp + page_or_route tag. `HOW` `scripts/perf_baseline.py` computes static via `radon cc` + line-count; `scripts/perf_capture_run.sh` runs Playwright against dev to harvest LCP/TBT/heap via `performance.timing` + Chrome DevTools Protocol. Cron orchestrator (`_task_perf_snapshot` at 04:00 IST) executes both; inserts row via `POST /api/admin/perf/snapshot` (internal, no auth required from cron). `WHERE` `backend/api/models.py::PerfSnapshot`, `backend/api/routes/admin.py`, `scripts/perf_*.py`.

Static metrics (per file)

Metric	Ideal	How	Notes
LOC	Components < 1500, Pages < 3000	`wc -l` (excludes blank + comment-only)	Baseline for cognitive load
cc_max	< 20	Cyclomatic complexity of most complex function in file via `radon cc`	Red flag at > 50: hard to maintain, high defect risk
cc_avg	< 8	Mean complexity across all functions	Safe zone; > 15 indicates refactor candidate

Metric	Ideal	How	Notes
$effectcount**\|<20perSveltefile\|Rege:$ **| Too many effects = hard to trace reactivity | | **$state count** \| < 50 per Svelte file \| Regex boundary match**$state(`declaratio:$** $cachebugs||**derived count**	Monitor trend	Regex boundary match `$derived` declarations	No hard limit; watch for explosion as sign of over-reactivity

Cyclomatic complexity thresholds (colour coding): - Green: `cc < 10` — safe, easy to read - Yellow: `cc 10–20` — moderately complex, watch carefully - Red: `cc > 20` — hard to maintain, high defect risk - Critical: `cc > 50` — refactor mandatory before merge

Runtime metrics (Web Vitals)

Metric	Ideal	How measured	Notes
LCP	< 2500 ms	Largest Contentful Paint — largest above-fold element render time	Chrome DevTools via Playwright
TBT	< 200 ms	Total Blocking Time — sum of long-task durations during page load	Chrome DevTools via Playwright
JS Heap	< 50 MB	V8 heap size after page settles (Chrome only)	Leak detector; baseline should not grow session-over-session
Route p50	< 200 ms	Median backend request latency (per endpoint)	Sampled from access logs
Route p95	< 500 ms	95th-percentile tail latency	Catches outlier slow requests
QPS	< 20 typical	Requests per second per route	Expected load during market hours

Data flow

Static compute — `scripts/perf_baseline.py --with-cyclomatic --no-build`: - Walks `backend/` + `frontend/src` recursively - For each `.py` file: LOC count + `radon cc` function list - For each `.svelte` file: LOC count + regex boundary match for `$effect(`, `$state(`, `$derived` declarations (regex patterns in the script handle `$derived.by(...)` and `$props({...})`) - Outputs JSON with per-file metrics

Runtime capture — `scripts/perf_capture_run.sh`: - Spins up Playwright against `dev.ramboq.com` - For each route in a curated list (positions, holdings, orders, dashboard, etc): - Navigate wait for settle (3s idle) capture `window.performance.timing.navigationStart`, `performance.memory.usedJSHeapSize` (Chrome only) - Extract LCP via `PerformanceObserver` + `'largest-contentful-paint'` entrypoint - Sum `performance.getEntriesByType('longtask')` duration for TBT - Outputs JSON per route

Nightly persist — `_task_perf_snapshot` (04:00 IST): 1. Calls `perf_baseline.py --with-cyclomatic --no-build` 2. Calls `perf_capture_run.sh` 3. Merges static + runtime JSONs 4. `POST /api/admin/perf/snapshot` with `{page_or_route, static_metrics, runtime_metrics}` 5. Inserts `PerfSnapshot(captured_at=now(), page_or_route=..., metrics_json=..., static_metrics_json=...)` row

DB schema

```
class PerfSnapshot(Base):
    __tablename__ = "perf_snapshots"

    id                  : int (PK)
    page_or_route       : str (e.g. "GET /api/positions", "/orders", "/dashboard")
    metrics_json        : dict (JSONB — runtime metrics: lcp_ms, tbt_ms, heap_mb, qps, ...)
    static_metrics_json : dict (JSONB — static metrics: loc, cc_max, cc_avg, ...)
    captured_at         : datetime (UTC, unique per (page_or_route, day))

    __table_args__ = (
        Index("ix_perf_snapshots_route_ts", page_or_route, captured_at.desc()),
    )
```

Retention: 365 days (config: `retention.perf_snapshots_days`, default 365).

Admin endpoints

- `GET /api/admin/perf/latest` — latest snapshot per page/route (no history).
- `GET /api/admin/perf/history?page=<route>&days=30` — time series for a single page_or_route over N days. Returns `[{captured_at, metrics_json, static_metrics_json}]`.
- `GET /api/admin/perf/regressions?days=7&threshold_pct=10` — detect pages where any metric exceeded 7-day median by > 10%. Returns list with regression alert.
- `POST /api/admin/perf/snapshot` — internal cron endpoint. No auth; HMAC signature (`X-PERF-SIGNATURE` header, signed with `api_secret`) required.

All endpoints are `admin`-guarded except `/snapshot`.

Frontend surface

`/admin/perf` dashboard — card grid layout: - **Code Health Card** — time-series chart of cc_max + LOC trend (30-day window) - **Runtime Card** — time-series chart of LCP + TBT + heap (30-day window) - **Regression Alert** — sticky callout at top if regressions detected in last 7 days - **Metrics Glossary** — info tooltips (hover on metric label show WHAT/IDEAL/IMPACT/FIX)

Metric metadata SSOT — `frontend/src/lib/data/metricMetadata.js` exports `METRIC_META`:

```
export const METRIC_META = {
  cc_max: {
    WHAT: "Highest cyclomatic complexity of any function in the file",
    IDEAL: "< 20 for maintainability",
    IMPACT: "High cc means more defects, longer review cycles",
    FIX: "Extract complex functions into helpers, simplify boolean logic"
  },
  lcp_ms: {
    WHAT: "Largest Contentful Paint — largest above-fold element render time",
    IDEAL: "< 2500 ms",
    IMPACT: "Poor LCP = operator sees blank page for seconds",
    FIX: "Lazy-load off-viewport components, defer non-critical JS"
  },
  // ... per metric
};
```

InfoHint component (reused everywhere) displays this tooltip on hover.

`#major` tag workflow

When operator adds `#major` prefix to any ask, the concurrent refactor flow is triggered:

1. Audit agent fetches top-3 cyclomatic/LOC hotspots from last 30 days of `perf_snapshots`
2. Identify regressions flagged in last 7 days
3. Parallel refactor tasks per hotspot
4. Recompute metrics after changes + assert no regression vs baseline

Example:

```
#major add a new route

→ audit sees cc_max creeping from 18→22 in chase.py over 2w
→ refactor: extract _chase_place_order, _chase_poll, _chase_status into focused
  functions (break one 60-line function into 3 functions × ~20 lines each)
→ recompute: cc_max now 14, tests pass
→ merge
```

Key files

- `backend/api/models.py::PerfSnapshot` — schema
- `backend/api/routes/admin.py::perf_snapshot` — cron endpoint
- `scripts/perf_baseline.py` — static metrics compute
- `scripts/perf_capture_run.sh` — runtime capture via Playwright
- `backend/api/background.py::_task_perf_snapshot` — orchestrator (04:00 IST)
- `frontend/src/lib/data/metricMetadata.js` — METRIC_META glossary
- `frontend/src/routes/(algo)/admin/perf/+page.svelte` — dashboard

Part II — Order lifecycle

5. Order placement — single ticket (Ticket tab)

```
sequenceDiagram
    actor OP as Operator
    participant OT as OrderTicket.svelte
    participant API as /api/orders/ticket
    participant DB as algo_orders
    participant CH as chase_order (background)
    participant BR as Broker
    participant PB as postback (Kite only)

    OP->>OT: Fill form + Submit
    OT->>API: POST /ticket (mode, side, sym, qty, price, template_id)
    API->>DB: INSERT AlgoOrder (status=OPEN, broker_order_id=NULL)
    API->>CH: place_order → chase_order (async)
    CH->>BR: place_order
```

```
    BR-->>CH: order_id
    API->>DB: UPDATE broker_order_id
    API-->>OT: success
    BR-->>PB: postback (Kite only)
    PB->>DB: UPDATE status + fill_price
    PB->>PB: _fire_template_attach_on_fill
```

Files: `frontend/src/lib/order/OrderTicket.svelte` (submit) → `backend/api/routes/orders.py::ticket_order` → `backend/api/algo/chase.py::chase_order` → postback (Kite) or polling.

Race-window guard: AlgoOrder commits with `broker_order_id=NULL` first. Fast IOC fill in this window is caught by postback-fallback matching `(account, symbol, side, qty, status=OPEN)` within 60s.

---

6. Order placement — basket (Chain tab)

Multi-leg submission grouped per-account, dispatched in parallel via `asyncio.gather`:

```
sequenceDiagram
    participant SP as SymbolPanel.svelte
    participant API as /api/orders/basket
    participant BR1 as Account A broker
    participant BR2 as Account B broker
    participant DB as algo_orders

    SP->>SP: Group basketLegs by account
    SP->>API: POST /basket (groups with legs)
    par dispatch A
        API->>BR1: place_order per leg
        BR1-->>DB: INSERT AlgoOrder per leg
    and dispatch B
        API->>BR2: place_order per leg
        BR2-->>DB: INSERT AlgoOrder per leg
    end
    API-->>SP: results (ok/fail counts)
```

Files: `frontend/src/lib/SymbolPanel.svelte::submitBasket` (grouping) → `backend/api/routes/orders.py::place_basket` (parallel dispatch).

Template isolation: legs with explicit `template_id` ignore shell overrides. Legs with no `template_id` inherit shell defaults.

---

7. Chase loop lifecycle

```
stateDiagram-v2
    [*] --> Placing
    Placing --> Polling: place_order returns order_id
    Polling --> Placing: status=OPEN AND price moved → cancel + replace
    Polling --> Filled: status=COMPLETE
    Polling --> Partial: cumulative_filled > already_filled
```

```
    Partial --> Polling: still has residual
    Partial --> Filled: cumulative = total
    Polling --> Rejected: status=REJECTED
    Polling --> KilledMidReplace: is_killed(NEW_id) post-replace
    Polling --> Killed: operator kill detected at status check
    Polling --> Unfilled: attempts >= max_attempts
    Polling --> ErrorAbort: >= _MAX_CHASE_ERRORS consecutive
    Filled --> [*]: _emit_chase_terminal(chase_fill)
    Rejected --> [*]: _emit_chase_terminal(chase_failed)
    Killed --> [*]: _emit_chase_terminal(chase_cancelled)
    KilledMidReplace --> [*]: _emit_chase_terminal(chase_cancelled, post-replace)
    Unfilled --> [*]: _emit_chase_terminal(chase_unfilled)
    ErrorAbort --> [*]
```

Key files: - `backend/api/algo/chase.py::chase_order` — main loop - `backend/api/algo/chase.py` — partial-fill branch (search `_record_partial_fill`) - `backend/api/algo/chase.py` — kill-race post-replace check (search `is_killed(current_order_id)` after `_sync_algo_order_id`) - `backend/api/algo/chase.py::_emit_chase_terminal` — snapshot + downstream attach - `backend/api/algo/chase.py::_sync_algo_order_id` — writes `broker_order_id` + `current_limit`

TECH — sync polling vs WebSocket order updates — `WHY` Postback delivery is unreliable for non-Kite brokers (Dhan + Groww are poll-only). Sync polling is the lowest-common-denominator that works everywhere. `WHAT` `chase_order` calls `_order_status` every 20s (configurable per chase). `HOW` Each iteration: depth quote adjusted limit cancel old + place new sync ID wait poll status. `WHERE` `backend/api/algo/chase.py`.

Partial-fill math (post C-1 fix):

```
already_filled = quantity - remaining_qty
new_delta = cumulative_filled - already_filled
fire partial branch when: cumulative_filled > 0 AND new_delta > 0 AND cumulative_filled < quantity
```

---

8. The order/chase/template tripod

This is the most complex part of the codebase. Three subsystems with overlapping responsibilities:

8.1 What each one owns

	Owns	Reads
Order routing (`orders.py`)	The broker-facing entry path. Single ticket + basket.	Settings, templates
Chase loop (`chase.py`)	The per-order placement lifecycle. Cancel + replace + status polling + partial fill accounting.	Broker, AlgoOrder, kill signal
Template attach (`template_attach.py`)	Post-fill exit-rule wiring. TP/SL/Wing/Scale/Trail GTTs at the broker.	OrderTemplate, AlgoOrder (read-only at attach time)

8.2 Why three? Why not one big “manage this order” function?

History: the chase loop existed first (single ticket place + chase to fill). Templates were bolted on later (Phase 0–3 + Sprints A–E). The current shape is intentional — each subsystem can be tested in isolation:

- Chase tests use a mock `_order_status` that returns a scripted sequence.
- Template tests build a `TemplatePlan` directly and assert the GTT spec shape.
- Routes are integration-tested with real broker mocks (`backend/tests/`).

8.3 The mode pivot

`mode ∈ {sim, paper, live, shadow}` decides which adapter the order actually hits. The pivot happens at submit time (`_resolve_mode` in `backend/api/algo/actions.py`) and is **persisted on the AlgoOrder row** — every downstream branch (chase terminal, postback, reconcile, template attach) reads `row.mode` to decide whether to call a real broker or the paper engine.

Gotcha: the chase loop runs the same code regardless of mode. Paper mode is achieved by injecting the paper engine's `place_order` adapter at the broker registry boundary. Don't add `if mode == 'live'` branches inside chase — the abstraction is the broker registry, not the chase.

Part III — Templates + exits

9. Template attach pipeline

```
flowchart TD
    subgraph triggers [Fill triggers]
        PB[Postback handler<br/>orders.py order_postback]
        CT[Chase terminal<br/>chase.py _emit_chase_terminal]
        RC[Reconcile sweep<br/>orders.py reconcile_*]
        RT[Operator retry<br/>orders.py retry_template]
    end

    PB --> FF[_fire_template_attach_on_fill]
    CT --> FF
    RC --> FF
    RT --> APT[apply_template_to_order]

    FF -->|attached_gtts_json IS NULL guard| APT
    APT --> RP[resolve_template_plan]
    RP --> PLAN[TemplatePlan: gtts + wing]
    APT --> WS[_pick_wing_by_premium<br/>chain scan]
    PLAN --> GTT1[broker.place_gtt — TP]
    PLAN --> GTT2[broker.place_gtt — SL]
    PLAN --> GTT3[broker.place_gtt — scale-out N]
    WS --> WO[broker.place_order — wing leg]
    GTT1 --> AGG[Aggregate result.gtt_ids]
    GTT2 --> AGG
    GTT3 --> AGG
    WO --> AGG
    AGG -->|attached_gtts_json| DB[(algo_orders.attached_gtts_json)]
    AGG --> RES[AttachResult<br/>{ok, errors[], notes[]}]
```

Key files: - `backend/api/algo/template_attach.py::resolve_template_plan` — override merge + scope

resolution - `backend/api/algo/template_attach.py::_pick_wing_by_premium` — OI + spread filters - `backend/api/algo/template_attach.py::AttachResult` — return type (NOT `TemplateAttachResult` — the docs previously had this wrong) - `backend/api/routes/orders.py::_fire_template_attach_on_fill` — idempotency guard + persistence - `backend/api/routes/orders.py::retry_template` — manual re-fire path. Persists `attached_gtts_json` per H-7 + trail-stop scaffolding per Sc.5

Idempotency: `_get_template_attach_lock(parent_row_id)` + `attached_gtts_json IS NULL` check. Strong dict with 1h TTL after M-5 fix replaces the prior WeakValueDictionary.

TECH — JSON blob vs normalized table for `attached_gtts_json` — `WHY` Each parent has 1-5 GTTs + maybe a wing. A child table would mean a JOIN on every order grid render. The blob lets us read the whole bracket in one column-fetch. `WHAT` Stored as a JSON array of entries (`{kind: "gtt", label: "TP", id: "...", current_trigger: ..., sl_trail_pct: ...}`). `HOW` Always write atomically (single column update); read+parse on every access (cheap because rows are small). `WHERE` `backend/api/models.py::AlgoOrder.attached_gtts_json` + `_fire_template_attach_on_fill`.

---

10. 4-default template matrix

```
flowchart LR
    Op([Operator picks symbol + side]) --> SC{_appliesToFor}
    SC -->|BUY + ends CE/PE| BO[buy_option<br/>default-long-option<br/>TP+80% MARKET]
    SC -->|BUY + EQ/FUT| BA[buy_any<br/>default-bull<br/>TP+30% SL-20%]
    SC -->|SELL + ends CE/PE| SO[sell_option<br/>default-short-vol<br/>TP+50% + Wing 10%]
    SC -->|SELL + EQ/FUT| SA[sell_any<br/>default-bear<br/>TP+30% SL-20%]
    BO --> CHIP[Default pill name chip<br/>+ override inputs]
    BA --> CHIP
    SO --> CHIP
    SA --> CHIP
    CHIP --> PREV[On-fill preview chip<br/>  triggers]
```

Key files: - `backend/api/algo/templates_seed.py::SYSTEM_TEMPLATES` — seeded defaults + rebalance logic - `frontend/src/lib/order/OrderTicket.svelte::_appliesToFor` - `frontend/src/lib/SymbolPanel.svelte::_appliesToFor` — same helper at shell level - `frontend/src/lib/SymbolPanel.svelte::_sideAwareDefault` — with fallback to focused-leg symbol - `frontend/src/routes/(algo)/automation/templates/+page.svelte` — coverage view (note: `/automation/templates` is the actual route; older docs reference `/admin/templates` which is stale)

---

11. Template override merge

Operator overrides flow through multiple layers; understanding the merge order saves hours of debugging.

```
Request:
  template_id=T1, tp_pct_override=20, sl_pct_override=None

Backend persist (orders.py::_build_overrides_json ~line 541):
  AlgoOrder.template_id          = T1
  AlgoOrder.template_overrides_json = '{"tp_pct": 20}'   # only NON-None overrides
```

```
At fill (template_attach.py::_pick):
  tp_pct = _ov.get("tp_pct") or template.get("tp_pct")
       → 20 (override wins)
  sl_pct = _ov.get("sl_pct") or template.get("sl_pct")
       → template's saved sl_pct (no override)
```

Per-leg vs shell: when a basket leg has `template_id` set explicitly (not inherited from `_sharedTemplateId`), the SHELL overrides DO NOT flow through. This is the audit-Sc.12 fix — pre-fix the shell's `tp_pct_override` silently contaminated a leg that the operator had retargeted to a different template.

```
submitBasket logic:
  effective_template = leg.template_id ?? shell_template
  if leg has its own template_id:
      tp_override = leg.tp_pct_override  (do NOT fall through to shell)
  else:
      tp_override = leg.tp_pct_override ?? shell.tp_pct_override
```

---

12. Chase loop invariants

Six things the chase loop MUST guarantee:

1. **AlgoOrder.broker_order_id always matches the LATEST broker order.** Cancel-and-replace updates this via `_sync_algo_order_id`. Without it the postback handler can't resolve a row by `broker_order_id`.
2. **AlgoOrder.current_limit reflects the latest re-quoted limit.** Added in M-6. ChaseCard renders this when present so the operator sees the live limit, not entry.
3. **AlgoOrder.filled_quantity is monotonic and never exceeds AlgoOrder.quantity.** The `MAX(prior, cumulative)` clamp in `_record_partial_fill` enforces this post C-1 fix. Template attach reads `filled_quantity` to size exit GTTs.
4. **Operator kills take effect on the very next loop iteration.** `mark_killed(broker_order_id)` is synchronous + the loop checks `is_killed(current_order_id)` (a) at status-check time AND (b) immediately after replace (C-2 fix). The dict has a 60-min TTL so a stale kill flag can't survive across days.
5. **Partial fills get persisted on every NEW delta, not just the first.** The branch fires when `cumulative > already_filled` post-C-1 fix.
6. **A chase that hits >= `_MAX_CHASE_ERRORS` consecutive exceptions aborts.** Prevents infinite re-trying against a broker that's down.

Break any of these and template attach sizes wrong, kills get ignored, or zombie chases burn rate limit.

---

13. Trail-stop subsystem

```
sequenceDiagram
    participant T as _task_trail_stop<br/>(every 30s)
    participant DB as algo_orders.attached_gtts_json
    participant PB as PriceBroker.ltp
    participant BR as broker.modify_gtt
```

```
    loop every templates.trail_poll_interval_seconds
        T->>DB: SELECT rows with sl_trail_pct
        T->>T: Build (sym, account, exchange) batches
        T->>PB: PriceBroker.ltp(keys) — falls over per broker
        PB-->>T: {key: {last_price}}
       T->>T: For each entry: compute new trigger<br/>long: peak × (1 − trail%)<br/>short: trough × (1 + trail%)
        alt new_trigger more favorable than current
            T->>BR: broker.modify_gtt(gtt_id, trigger=new)
            alt modify succeeds
                BR-->>T: ok
                T->>DB: UPDATE attached_gtts_json with new trigger
            else Dhan partial (ENTRY_LEG ok, TARGET_LEG fail)
                BR-->>T: raise(dhan_partial_modify=True)
                T->>DB: persist partial_modify_error
                T->>T: WARNING log + Telegram alert
                T->>T: pop sl_trail_pct (stop ratcheting)
            else NotImplementedError
                T->>T: pop sl_trail_pct (stop ratcheting)
            end
        end
    end
```

Key files: - `backend/api/background.py::_task_trail_stop` - `backend/api/background.py` — Dhan partial-modify detect + alert (M-2 fix, search `dhan_partial_modify`) - `backend/brokers/adapters/dhan.py::modify_gtt` — two-leg dispatch (Sprint C) - `backend/brokers/adapters/groww.py` — emulated OCO trail (currently `NotImplementedError`-skip) - `backend/brokers/adapters/dhan.py::ltp` — wired via instruments cache (B-2 fix)

Asymmetric SELL guard note: the poller's SELL ratchet check is `current_trigger > 0 AND proposed < current_trigger`. If `current_trigger=0` (entry never persisted), the guard short-circuits □ trail silently dead. This is why **every persistence path that writes a trail entry MUST seed `current_trigger`** (see Sc.5a fix in `retry_template`).

Part IV — Brokers

14. Broker abstraction

```
flowchart TD
    subgraph routes [Route layer]
        OR[orders.py routes]
        AC[actions.py agent actions]
        BG[background.py tasks]
    end

    subgraph reg [Registry]
        GB[get_broker account]
        GPB[get_price_broker]
        GHB[get_historical_brokers<br/>Kite-only filter]
        GSB[get_sparkline_broker<br/>Kite-only filter]
```

```
    end

    OR --> GB
    AC --> GB
    BG --> GB
    OR --> GPB
    AC --> GPB
    BG --> GPB

    subgraph abc [Broker ABC]
        OABC[order_status]
        PABC[place_order]
        MABC[modify_order / cancel_order]
        GABC[place_gtt / modify_gtt / cancel_gtt]
        LABC[ltp / quote / historical_data]
    end

    GB --> KITE[KiteBroker]
    GB --> DHAN[DhanBroker]
    GB --> GROWW[GrowwBroker]

    KITE -.implements.-> abc
    DHAN -.implements.-> abc
    GROWW -.implements.-> abc

    KITE --> KSDK[kiteconnect SDK]
    DHAN --> DSDK[dhanhq SDK]
    GROWW --> GSDK[growwapi SDK]
```

Capability matrix surface: - `backend/brokers/capabilities.py::BrokerCapabilities` — dataclass with every capability flag - `backend/brokers/registry.py::get_historical_brokers` — Kite-only filter - `frontend/src/lib/data/brokerCapWarnings.js` — single source of truth for warning strings (H-5) - `frontend/src/lib/order/OrderTicket.svelte::capWarningFor` — single-account - `frontend/src/lib/SymbolPanel.svelte::aggregateCapWarnings` — cross-account (H-5)

 TECH — PriceBroker fallback chain — `WHY` Some brokers can answer quote/ltp/historical (Kite), some can't (Dhan returns `{}` by design for `quote`). Walking the chain lets the operator's chart still render even when their primary account is throttled. `WHAT` `PriceBroker._try(method_name, *args)` iterates eligible brokers, calls method, checks predicates (`_quote_has_data` / `_ltp_has_data` / `_historical_has_data`), returns first successful response. `HOW` Add a new method by name in the predicate map. Rate-limit cool-off (`_RATE_LIMIT_COOLOFF`) excludes throttled accounts for 30s. `WHERE` `backend/brokers/registry.py::PriceBroker`.

14.5. Broker abstraction — implementation detail

Full broker layer architecture — file map, singleton lifecycle, token caching, source-IP binding, and capability matrix — **lives in CLAUDE.md §14.5 for brevity**. This section is a quick read list only.

Files — `backend/brokers/{base.py, kite.py, dhan.py, groww.py, capabilities.py, registry.py}` + `backend/shared/helpers/{connections.py, broker_creds.py, kite_ticker.py}`.

Key rules: 1. **Kite-shape contract** — every return value must match Kite Connect shape. Dhan/Groww

adapters have `_normalise_*` helpers. The `_DHAN_STATUS_TO_KITE` status map is critical (audit B-1). 2. **Singleton per process** — adapters live via `Connections()` singleton. Each Kite login takes 10-15s; re-doing per-request is unworkable. 3. **IPv6 source-binding** — Kite + Dhan enforce one-session-per-IP rules. Each account binds to a unique IPv6 via `_IPv6SourceAdapter` (Kite/Dhan) or ContextVar proxy (Groww). 4. **Token cache** — each broker persists tokens to `.log/<broker>_tokens.json`. On startup, skips login if fresh token cached; fires full login only on miss/expiry/manual delete. 5. **Registry factories** — use `get_broker(account)` for operator actions, `get_price_broker()` for shared market data, `get_historical_brokers()` for OHLCV + regression, `get_sparkline_broker()` for KiteTicker. 6. **Capabilities immutable** — frozen dataclass with every field explicit per broker (no defaults). Used to render warning chips on OrderTicket when template asks for unsupported GTT shape.

PriceBroker fallback chain — when a broker returns empty data (Dhan returns {} for MCX quotes by design), walk to the next broker. Rate-limit cool-off excludes throttled accounts for 30s. postback_gtt="reliable", rate_limit_orders_sec=10,) DHAN_CAPS = BrokerCapabilities(broker_id="dhan", display_name="Dhan", gtt_single=True, gtt_oco=True, gtt_modify=True, gtt_cap_per_account=50, gtt_validity_days=365, gtt_supports_mcx=False, bracket_order=True, cover_order=True, atomic_basket=True, order_tag=True, margin_preview=True, # Audit fix — no Dhan WebSocket / GTT-fire postback handler is wired # in the codebase today; detection is the poll-based _task_oco_pair_watcher postback_gtt="poll_only", rate_limit_orders_sec=20,) GROWW_CAPS = BrokerCapabilities(broker_id="groww", display_name="Groww", gtt_single=True, gtt_oco=False, gtt_modify=True, gtt_cap_per_account=25, gtt_validity_days=90, gtt_supports_mcx=False, bracket_order=False, cover_order=False, atomic_basket=False, order_tag=False, # No native tag; broker_order_link sidecar covers it margin_preview=False, postback_gtt="poll_only", rate_limit_orders_sec=5,)

```
OCO emulation for Groww (no `gtt_oco`) is implemented in `groww.py::place_gtt` via two single-
trigger GTTs + the `_task_oco_pair_watcher` background task that cancels the surviving sibling when one fires. There'

`CAPS_BY_BROKER_ID` maps both the canonical `"zerodha_kite"` and the legacy `"kite"` alias to `KITE_CAPS`, so older Y
seeded rows keep resolving without a column rewrite.

Frontend reads via `GET /api/admin/brokers/{account}/capabilities` (in-
memory, no broker call). UI helper `brokerCapWarnings.js` consults the matrix to warn the operator at submit time wher

### 14.5.9 Per-broker quirks worth knowing

These are documented inline in code, but listed here for orientation:

| Broker | Quirk | Where it's handled |
|---|---|---|
| **Kite** | `tag` is 20-char max | `_truncate_tag` in `backend/brokers/adapters/kite.py` |
| **Kite** | Postback HMAC validation | `order_postback` route checksum check |
| **Kite** | Rate-limited historical_data quota (low per-second budget) | `_RATE_LIMIT_COOLOFF` 30s window in registr
`_RATE_LIMIT_COOLOFF_SECONDS = 30` |
| **Kite** | One-IP-per-app rule | `_IPv6SourceAdapter` mount |
| **Dhan** | Token dashboard validity defaults to 5min | Operator must extend to 24h in Dhan dashboard |
| **Dhan** | `ltp()` returns `{}` for MCX commodity by design | PriceBroker fallback + B-
2 fix logs the empty response |
| **Dhan** | `modify_gtt` needs TWO calls (ENTRY_LEG + TARGET_LEG) | Sprint C dispatch in `dhan.py::modify_gtt` |
```

Dhan	One-active-token-per-app-per-IP rule	`_IPv6SourceAdapter` + multi-account stabilizer in `Connections`
Groww	Module-level `requests` calls with no session hook	ContextVar monkey-patch (see §14.5.5)
Groww	No native OCO	Emulated via two single GTTs + `_task_oco_pair_watcher` cancellation of survivor
Groww	`cancel_gtt` needs exchange (numeric id collision risk)	M-4 fix raises if exchange missing
Groww	No `historical_data` support	`historical_data=False` cap; sparkline + chart endpoints fall over to Ki

14.5.10 Modifying the broker layer — guard rails

If you're touching anything in `backend/brokers/`:

- **Never branch in callers by `broker_id`.** The whole point of the abstraction is that callers don't know which vend
- **Always re-shape vendor responses to Kite shape.** Frontend + chase + template all expect Kite shape. Skipping the
- **Status maps are non-negotiable.** Every vendor status must map to a Kite-canonical status (`COMPLETE`, `OPEN`, `CANCELLED`, `REJECTED`, `EXPIRED`, `TRIGGER PENDING`). A missing entry breaks
- **Wrap sync SDK calls in `asyncio.to_thread`.** Adapters are sync internally; callers MUST go through `asyncio.to_t
- **Log silent failures.** B-4 audit fix: when a broker SDK returns empty data instead of raising, log `WARNING` with m
- **Update `capabilities.py` first.** If you discover a vendor supports something we'd marked `False`, update the mat
visible warnings flow from the matrix.

---

15. How to add a new broker

If you're integrating a new vendor (e.g. "Upstox"):

Backend

1. **Implement the adapter** in `backend/brokers/upstox.py`. Subclass `Broker` (ABC at `base.py`). Implement EVERY me
there's no "partial" mode. If a method genuinely doesn't apply, raise `NotImplementedError` with a clear message rath

2. **Translate to Kite shape.** Every method that returns operator-facing data (orders, positions, ltp, GTTs) must shape its return to match Kite's structure. Frontend renders are Kite-
shape; downstream chase + template code expects Kite-shape. Build a `_normalise_*` helper per category. Mirror the pa

3. **Status-string normalization.** Add `_UPSTOX_STATUS_TO_KITE = {...}`. Every Kite-canonical status (`COMPLETE`, `OPEN`, `CANCELLED`, `REJECTED`, `EXPIRED`) must map from one Upstox string. The B-1 audit lesson: a single missing entry silently breaks chase fill detection for an entire broker.

4. **Capabilities.** Add `UPSTOX_CAPS` in `capabilities.py` with EVERY field set explicitly. Don't rely on dataclass being explicit makes capability gaps visible at code review.

5. **Register in `registry.py`.** Add to `_ADAPTERS` map + `CAPS_BY_BROKER_ID`.

6. **Token caching.** Each broker has its own `.log/<broker>_tokens.json`. Follow the connection wrapper pattern in `

7. **Tests.** Add `backend/tests/test_upstox_broker.py`. Mock the vendor SDK at the boundary; assert your `_normalise

Frontend

8. **No frontend code change needed.** The `BrokerCapabilities` dataclass is the contract; the cap warning helper at `
visible capabilities surface automatically.

---

16. Broker gotchas

Documented so you don't relearn them the hard way:

Gotcha	Bit us in
Postback arrives before broker_order_id is committed	Race window between AlgoOrder pre-persist + seed-broker_id second commit. Fix: fallback recent-NULL-id match (C-3)
Cumulative vs delta in status polls	Every broker reports `filled_quantity` cumulatively. Prefix we added the cumulative value each call → inflation across restarts (C-1)
Kill recorded against old broker_order_id	Cancel-and-replace creates a new id; kill was only checked against old. Operator's kill silently ignored (C-2)
WeakValueDictionary GC during await	`_TEMPLATE_ATTACH_LOCKS` could be GC'd between mint and acquire. Fix: str
5). 1h chosen because longest realistic live-chase window is ~30 min; 1h is 2× headroom
Reconcile attach BEFORE commit	Attach pipeline opened its own session and read pre-commit state. Fix: defer to after commit (C-4 single + bulk)
Empty `_normalise_orders` status map	Groww's "EXECUTED" passed through verbatim; chase loop never saw "COMPLE
1)
Dhan `ltp()` returned `{}` by design until B-2. Trail stop silently dead — no log, no Telegram, just zero ratchet	
Groww `cancel_gtt` blind segment fallback	Could cancel wrong GTT on numeric id collision. Now raises if exchan
4)
Naive `datetime.now()` in DB writes	Mix with tz-aware columns → "AT TIME ZONE" errors. Always `datetime.now(timezone.utc)`
Kite's `tag` is 20-char max	We truncate via `_truncate_tag` in `chase.py`
Trail-stop persistence missing `current_trigger`	Asymmetric SELL guard short-circuits with `0`. Every trail-write path must seed (Sc.5a)
OCO double-fire 15s window	`oco_pair_poll_seconds` default. Matches the `poll_only` GTT detection lag operatc
8)
60-second postback fallback window	Long enough to cover the slowest IOC fill + DB commit race; short enough to pollination with new orders (C-3)

Part V — Frontend

17. Frontend modal state

```mermaid
flowchart TD
    SP[SymbolPanel.svelte]
    SP -->|tab=ticket| OT[OrderTicket.svelte]
    SP -->|tab=chain| OCT[OptionChainTab.svelte]
    SP --> TPL[Template row: Default/None pill]
    SP --> BB[Basket bar pills]
    SP --> CC[ChaseCard.svelte]

    OT --> OD[OrderDepth.svelte]
    OT -->|onMarginUpdate| SP
    OT -->|onPreviewPlanUpdate| SP

    subgraph shellState [Shell-level state]
        SA[_sharedAccount]
        ST[_sharedTemplateId]
        SO[_sharedTpOverride / Sl / Wing×2]
        BL[basketLegs[]]
        FK[_focusedLegKey]
    end

    SP -.binds.-> SA
    SP -.binds.-> ST
    SP -.binds.-> SO
    SP -.owns.-> BL
    SP -.owns.-> FK

    OT -.binds.-> SA
    OT -.binds.-> ST
    OT -.binds.-> SO

    OCT -.binds.-> SA
    OCT -.binds.-> ST
    OCT -.onAddLeg.-> BL

    TPL -.reads.-> ST
    BB -.iterates.-> BL
    BB -.click pill.-> FK
```

Key files: - `frontend/src/lib/SymbolPanel.svelte` — shell + Template row + basket bar + chase card mount - `frontend/src/lib/order/OrderTicket.svelte` — Ticket form + depth ladder + margin preview - `frontend/src/lib/order/OptionChainTab.svelte` — strike grid + futures + chain quotes - `frontend/src/lib/order/OrderDepth.svelte` — bid/ask depth (visibility-gated polling)

Preview chip swap rule (Chain tab): - `basketLegs.length === 0` → Ticket-form preview - `basketLegs.length > 0` + no focus → last-leg preview - `_focusedLegKey != null` → that specific leg's preview, badge shows `LEG N/M` ● - Click any basket pill → set `_focusedLegKey` - Click chip itself → cycle to next leg - Operator × on focused leg → key clears, falls back to last-leg

 TECH — Svelte 5 `$bindable()` props — `WHY` Two-way sync without prop-drilling or a global store. `WHAT` Child component declares `let { templateId = $bindable(null) } = $props();` parent writes `bind:templateId={_sharedTemplateId}`. `HOW` Mutations on either side propagate. Avoid `bind:` for derived values (use a `$derived` instead). `WHERE` `SymbolPanel.svelte` `OrderTicket.svelte` `OptionChainTab.svelte` template + account props.

---

18. Frontend state architecture

18.1 Why no global store for order state?

Svelte stores would be the obvious pattern but we don't use them for order modal state. Reasons:

- **Modals are short-lived.** The operator opens, fills, submits, closes. State outlives a single modal mount maybe 5% of the time (basket persists across tab flips).
- **Component-local state with bindable props is enough.** `bind:value` on Svelte 5 runes provides bidirectional sync without the boilerplate.
- **One modal at a time.** We don't need a global "current order context" — the modal owns its context.

The exceptions: `executionMode` (navbar drives every page), `authStore` (every page), `dataCache` (PositionStrip + dashboards share), `orderTemplatesStore` (template CRUD broadcast). These are all narrow — they don't carry order-specific state.

18.2 The "shell" pattern

`SymbolPanel.svelte` is a shell. It owns: - Header (account + symbol pickers) - Tabs (Ticket / Chain) - Template row (Default/None pill + override inputs + preview chip) - Basket bar (when basket has legs) - Common action footer (margin chip + Submit)

The actual tab content (`OrderTicket.svelte`, `OptionChainTab.svelte`) is mounted as a child. State pipes through: - **Down via props:** shell tab (e.g. `_sharedAccount` `account` prop) - **Up via callbacks:** tab shell (e.g. `onMarginUpdate`, `onPreviewPlanUpdate`) - **Two-way via `bind:`:** for shared mutable state (e.g. `bind:templateId={_sharedTemplateId}`)

When you add a new piece of shell-visible state, decide once: - Is it tab-specific? Stay in the tab component. - Should it survive tab flips? Lift to shell. - Does any tab need to READ it? Pipe down via prop. - Does any tab need to WRITE it? `bind:` it.

---

19. The preview pipeline

The on-fill preview chip (`on fill → TP  250 / SL  180 / + Wing BUY ...`) is the single most useful piece of context at submit time. It's computed via two independent pipelines:

- **OrderTicket's `_previewPlan`** computed against the Ticket form's symbol/side/qty/price/template.
- **SymbolPanel's `_lastLegPlan`** computed against the last basket leg (or operator-focused leg).

The chip render switches between them based on `_activeTab === 'chain' && basketLegs.length > 0`. **Why two pipelines?** Because the inputs are different: - Ticket: form state, not yet a "leg" - Last-leg: a fully-formed leg with its own account + symbol + overrides

Both call the same backend endpoint (`previewTicketTemplate`) with the same payload shape. The frontend just feeds them differently.

 TECH — Backend preview endpoint vs frontend simulation — WHY Operators trust values that come from the same code path that will ACTUALLY fire on fill. Computing them in the frontend would risk drift; computing them in the backend guarantees the chip reflects reality. WHAT `POST /api/orders/preview-ticket-template` returns `{plan: {gtts: [...], wing: {...}, notes: [...]}}`. HOW Frontend debounces 200ms after any override change; backend runs `resolve_template_plan` with `apply_path="preview"` so no broker calls fire. WHERE `backend/api/routes/orders.py::preview_ticket_template`.

Part VI — Runtime

20. Background task topology

```
gantt
    title Background tasks (app.on_startup)
    dateFormat HH:mm
    axisFormat %H:%M
    section Market data
    Performance refresh (5min)  :perf, 09:00, 6h
    Sparkline warm (daily 00:30) :spark, 00:30, 1m
    Hedge proxy regression (daily 02:30) :hp, 02:30, 5m
    section Order lifecycle
    OCO pair watcher (15s)      :oco, 09:00, 6h
    Trail-stop poller (30s)     :trail, 09:00, 6h
    Ticker watchdog (30s)       :tw, 09:00, 6h
    section Daily ops
    Open summaries              :open, 09:15, 5m
    Close summaries             :close, 15:30, 5m
    MCP audit cleanup (03:15)   :mcp, 03:15, 1m
```

Key files: - `backend/api/background.py` — all task definitions - `backend/api/app.py::on_startup` — spawn list

Tasks that touch operator orders: - `_task_performance` (5min) — fetches positions/holdings/funds; runs `agent_engine.run_cycle` - `_task_oco_pair_watcher` (15s) — Groww emulated OCO sibling cancel - `_task_trail_stop` (30s) — Dhan + Kite trail SL ratchet - `_task_ticker_watchdog` (30s) — KiteTicker reconnect on disconnect

 TECH — Why poll-based + not event-based — WHY Vendor postbacks are unreliable (Dhan + Groww have no inbound webhook; Kite drops 0.5-2% in our experience). Polling is the conservative floor. WHAT Each task runs on its own asyncio cadence; no scheduler library. HOW Pick interval based on operator latency tolerance: trail-stop = 30s (slow ratchet OK), OCO watcher = 15s (faster because both legs settling within window means double-fire). WHERE `backend/api/background.py`.

---

21. Data refresh — PositionStrip + Dashboard

```
sequenceDiagram
    actor OP as Operator
    participant PS as PositionStrip.svelte
```

```
    participant API as /api/positions, /api/holdings, /api/funds
    participant BR as Broker (Kite)
    participant CACHE as dataCache (in-memory)

    OP->>PS: Mount on any algo page
    PS->>CACHE: Read last-good snapshot for fast paint
    PS->>API: fetchPositions + fetchHoldings + fetchFunds (parallel)
    API->>BR: kite.positions + kite.holdings + kite.margins
    BR-->>API: rows
    API-->>PS: rows
    PS->>CACHE: Update dataCache
    loop every 30s (marketAwareInterval)
        PS->>API: re-fetch
    end
    note over PS: positionsPnl = sum(p.pnl)<br/>positionsToday = sum(p.day_change_val)<br/>holdingsToday =
sum(h.day_change_val)<br/>holdingsTotal = sum(h.pnl)
```

Key files: - `frontend/src/lib/PositionStrip.svelte` — navbar strip aggregations - `backend/api/routes/positions.py`, `holdings.py`, `funds.py` — REST endpoints - `backend/brokers/broker_apis.py::fetch_positions / fetch_holdings / fetch_margins` - `backend/api/cache.py` — server-side cache (per-key locking + TTL)

`/admin/derivatives` Snapshot TOTAL reconciles to PositionStrip by adding back the rows the page filters out (equity intraday positions + derivative-looking holdings) via `_excludedByAccount`. See `frontend/src/routes/(algo)/admin/derivatives/+page.svelte` (search `_byUnderlyingTotal`).

TECH — `marketAwareInterval` polling vs WebSocket — `WHY` Position state changes when fills happen; we already get fills via KiteTicker, but positions are aggregated server-side. Polling is cheaper than rebuilding aggregations client-side. `WHAT marketAwareInterval(fn, 30000)` polls every 30s during market hours, pauses on `document.hidden`. `HOW` Use the helper from `$lib/stores`; never raw `setInterval`. `WHERE frontend/src/lib/stores.js::marketAwareInterval`.

---

22. Demo mode

```
flowchart LR
    Anon([Anonymous prod visitor]) --> AUTH{authStore.user}
    AUTH -->|null + branch=main| DEMO[Demo session<br/>state.is_demo = True]
    AUTH -->|signed in| AUTHED[Authenticated session]

    DEMO --> RB[Read paths: real data, accounts masked<br/>ZG0790 → ZG####]
    DEMO --> WB[Write paths: blocked at API]
    WB -->|POST /orders/place| 403
    WB -->|POST /orders/ticket mode=live| DOWNGRADE[Silently downgraded to paper]
    WB -->|/api/admin/*| 401

    DEMO --> UI[UI shows:<br/>· Sign In button replaces user pill<br/>· Settings/Brokers/Users hidden<br/>·
Template picker shows muted note]
```

Key files: - `backend/api/auth_guard.py::is_demo_request` + `auth_or_demo_guard` - `frontend/src/routes/(algo)/+layout.svelte` — demo nav-link gating - `frontend/src/lib/SymbolPanel.svelte` —

template row demo gate (L-3) - `backend/brokers/broker_apis.py::mask_column` — for demo + public

22.5. Investor portal — token-as-credential

Public read-only NAV surface for LPs. The URL `/investor/<token>` IS the credential — no login, no password. Operator mints from `/admin` per-user Portal button, copies the URL, forwards through their own channel (WhatsApp / email).

Why token-as-credential

The boutique fund has 1–5 LPs. Asking each LP to manage a password for a quarterly NAV check is friction nobody wants. Carta, SS&C/GP-Link, and Yieldstreet all converge on the same pattern for LP-facing statements: a long-lived per-LP URL that's revocable on suspicion of leakage. Same threat model as a long-lived API key — if you trust the recipient with the URL, the URL is fine.

 TECH — Long-lived URL token vs JWT magic-link — `WHY` JWT magic-links are short-lived (5-15 min); they're great for a one-time "log in to this session" handshake but useless for "bookmark this URL and re-check the value every Friday." The investor portal is a recurring read-only surface, not a session. `WHAT` 32-byte `secrets.token_hex` 64-char string, 90-day default expiry, revocable. `HOW` Stored raw in `investor_tokens.token` (same convention as `AuthToken` — the token IS the URL slug; hashing adds no security since possession of the URL == access). `WHERE` `backend/api/models.py::InvestorToken`, `backend/api/routes/investor.py`.

Schema

```
class InvestorToken(Base):
    __tablename__ = "investor_tokens"
    id, user_id (FK users.id), token (64-char unique),
    expires_at, revoked_at (nullable),
    last_visit_at, visit_count (operator visibility),
    note (admin label), created_by (FK users.id), created_at
```

`Base.metadata.create_all` picks it up on next deploy — no migration.

Active check

`_is_active(row, now) := row.revoked_at is None AND row.expires_at > now`. Three terminal states surfaced in the admin UI: ACTIVE (green) / REVOKED (red) / EXPIRED (slate).

Endpoints

Admin (`manage_investor_tokens` cap, admin-only): - `GET /api/admin/users/{id}/investor-tokens` — list (preview only, never full token) - `POST /api/admin/users/{id}/investor-tokens` — mint (returns full token + portal URL ONCE) - `DELETE /api/admin/users/{id}/investor-tokens/{tid}` — revoke

Public (no auth — token in URL is the credential): - `GET /api/investor/{token}/slice` `InvestorSliceResponse` (same math as `/api/nav/me`) - `GET /api/investor/{token}/history?days=180` curve

Visit tracking

`_resolve_token()` bumps `last_visit_at` + `visit_count` on every successful resolve via a best-effort `UPDATE` (try/except, rollback on failure). The operator's admin modal surfaces the timestamp + count so they know "this LP last looked 3 weeks ago" without leaving the page. The counter increments per endpoint hit, so a single

page load bumps it by 2 (slice + history).

Frontend separation

The portal page lives at `frontend/src/routes/investor/[token]/+page.svelte` — sibling of `(public)` and `(algo)` route groups. It inherits only the root `+layout.svelte` (which is empty — just `{@render children()}`), so it gets none of the algo navbar or the public marketing nav. Cream + champagne palette matching the marketing site so LPs land on a "professional statement" page, not a Bloomberg-style trading desk.

Robots `noindex,nofollow` in `<svelte:head>` so leaked URLs don't end up in search engines.

Revocation model

Revoke is destructive but trivially reversible — admin clicks Revoke (confirms via `ConfirmModal`), `revoked_at` flips to `now()`, next visit 401s. Re-mint creates a new row; the old row stays for the audit trail.

Idempotent: revoking an already-revoked row is a no-op (the original `revoked_at` timestamp is preserved so "when did we revoke this?" remains accurate).

Source files

- `backend/api/models.py::InvestorToken`
- `backend/api/routes/investor.py` — both controllers
- `backend/api/rbac.py::CAPS["manage_investor_tokens"]`
- `frontend/src/routes/investor/[token]/+page.svelte` — LP-facing page
- `frontend/src/routes/(algo)/admin/+page.svelte` — `openPortal()` modal + mint flow
- `frontend/src/lib/api.js::{fetchInvestorTokens, mintInvestorToken, revokeInvestorToken}`

22.6. Investor portal — units-based NAV math

The v1 model (`slice = share_pct × firm_nav`) is **retired**. Every LP slice / cost-basis / P&L computation now flows through the standard fund-accounting units model:

```
units_held(user, t)   = Σ units_delta for user's events <= t
total_units(t)        = Σ units_held across every LP
nav_per_unit(t)       = firm_nav(t) / total_units(t)
slice(user, t)        = units_held × nav_per_unit
cost_basis(user, t)   = Σ amount (sub+bootstrap) − Σ amount (redemption)
pnl(user, t)          = slice − cost_basis
```

 TECH — Units vs static share_pct — `WHY` static share_pct breaks when an LP joins mid-period (their cost basis is the day they bought in, not "since fund inception") or partially redeems (the remaining slice's basis must shrink in proportion). `WHAT` Each LP holds a count of partnership units; the fund publishes a per-unit value daily; slices are products. `HOW` Subscription buys units at the day's nav_per_unit, redemption sells at the day's nav_per_unit, gains accrue automatically through firm_nav growth. `WHERE backend/api/algo/investor_units.py` is the single math source; four callsites consume it.

Single source of math

`backend/api/algo/investor_units.py` exposes:

- `ensure_user_bootstrap(s, user)` — idempotent synthetic-event seed for v1 units migration
- `ensure_all_bootstrapped(s)` — covers every eligible LP; called at the start of every units compute

- `units_held(events, as_of)` / `cost_basis(events, as_of)` — pure-function primitives
- `slice_value(user_events, all_events, firm_nav, as_of)` — returns `(slice, nav_per_unit)`
- `compute_slice(s, user, firm_nav, as_of)` — DB-aware wrapper
- `compute_slice_history(user_events, all_events, firm_curve)` — pre-fetched, walks dates in pure Python

Switched callsites (all four):

Surface	Old	New
`/api/nav/me` (authenticated LP)	`share_pct × firm_nav / 100`	`compute_slice()`
`/api/nav/me/history`	scaled curve	`compute_slice_history()`
`/api/investor/{token}/slice` + `/history` (public portal)	scaled curve	`compute_slice` / `compute_slice_history`
`compute_statement()` (monthly PDF)	period-anchored scale	event-walked slice

Auto-bootstrap rule

On every units compute, `ensure_all_bootstrapped(s)` scans every eligible LP (`is_active=True AND share_pct > 0`) and inserts a synthetic event for any without one. The bootstrap row encodes the v1 state:

```
InvestorEvent(
    event_type   = "bootstrap",
    units_delta  = user.share_pct,
    amount       = user.contribution,
    nav_per_unit = contribution / share_pct  # 1.0 fallback when contribution=0
    event_date   = contribution_date or created_at.date() or today,
)
```

This guarantees that **the first request after this code lands reproduces v1 numbers identically** when share_pcts sum to 100 across all eligible LPs. When the sum != 100 (operator-residual case with implicit ownership), the units model redistributes proportionally and slices sum to `firm_nav` by construction — slightly different numbers, but internally consistent + correct going forward.

Day-delta semantics under units

`day_delta_share` on `/api/nav/me` is computed as `slice(today) − slice(prior)`, both via the SAME event set. This means:

- A pure market gain shows up as positive day-delta
- A subscription between the two snapshots inflates `slice(today)` but ALSO appears in cost_basis, so it doesn't read as P&L on the LP's portal
- A redemption deflates `slice(today)` symmetrically

The portal UI labels this as "Day Δ" but the operator should read it as "change in your slice's market value since yesterday's snapshot."

Smoke-test invariants

Math is verified end-to-end against a fixture (see commit `322f0c22`):

Scenario	Invariant	Verified
All LPs at bootstrap, share_pcts sum to 100	slices match v1 exactly	☐
Fund grows N%	each LP's slice grows N% on their basis	☐
LP_A subscribes mid-period at higher nav_per_unit	extra subscription doesn't double-count as P&L	☐
Any state	Σ slices == firm_nav (modulo rounding)	☐

Bootstrap edit / correction path

Operator wants to fix a bootstrap row (wrong contribution, wrong share_pct in `users` row, missing LP):

1. Edit `User.contribution` / `share_pct` in `/admin` (existing flow)
2. Delete the bootstrap event in `/admin` → Portal → Events tab
3. Next compute auto-bootstraps with the corrected User columns

The delete-then-recompute cycle preserves history (bootstrap event has the old timestamp) without manual reconciliation.

Source files

- `backend/api/algo/investor_units.py` — math + bootstrap
- `backend/api/algo/investor_statement.py::compute_statement` — PDF math via units
- `backend/api/routes/nav.py::my_slice` + `my_history` — authenticated LP endpoints
- `backend/api/routes/investor.py::slice` + `history` — public portal endpoints
- `backend/api/models.py::InvestorEvent` — events journal table

22.7. Audit log — forensic trail

Single `audit_log` table catches every mutating event the platform produces — HTTP mutations (via middleware), broker fills (via postback handler), agent-initiated actions (via the action dispatcher), and background-task events (NAV compute, monthly statement send). Read surface at `/admin/audit` is cap-gated to `view_audit` (designated / admin / risk).

☐ TECH — ASGI middleware + fire-and-forget writes — `WHY` SEBI Cat-III's "every mutating event" requirement can't be satisfied with per-route decorators (easy to forget; impossible to enforce). A middleware catches everything by default. `WHAT` `AuditMiddleware` wraps every HTTP response; on a mutating 2xx/3xx it schedules a `_write_audit` coroutine via `asyncio.create_task`. Response leaves the server immediately; the DB write lands shortly after. `HOW` Failed audit writes log a warning and drop — the user's request never blocks on the audit pipeline. `WHERE` `backend/api/audit.py`.

Schema

```
class AuditLog(Base):
    id, actor_user_id (FK users.id), actor_username, actor_role,
    action, category, method, path,
    target_type, target_id,
    status_code, summary,
    request_id (UUID, mirrored in X-Request-ID),
    client_ip, user_agent, created_at
```

- `actor_*` fields are SNAPSHOTTED — a later role demotion doesn't rewrite history.
- `category` is a coarse tag (`order.fill`, `agent.action`, `system.nav`, …) populated by `_derive_category_from_path` for HTTP rows and explicitly by `write_audit_event` callers.
- `request_id` correlates each audit row with the API log line for the same request.

Two write paths

1. HTTP middleware (default) — `AuditMiddleware.handle` watches every response. Skips non-mutating methods + `_SUPPRESS_PREFIXES`. Captures actor from JWT, status code from the wrapped `send`, body summary from the first 1 KB of response. Path-derived category via `_derive_category_from_path`.

2. Non-HTTP helper (added Jun 2026) — `write_audit_event(category, action, ...)` is the public API for any code path that mutates state without going through HTTP:

Caller	Category	Actor
Broker postback handler (`orders.py`)	`order.fill` / `order.cancel` / `order.reject`	`broker`
Agent action dispatcher (`actions.py::execute`)	`agent.action`	`agent:<slug>`
Monthly statement send (`background.py`)	`system.statement`	`system`
NAV compute (`background.py`)	`system.nav`	`system`

The helper is fire-and-forget (`asyncio.get_running_loop().create_task(...)`); failed writes log a warning and drop. Sim-mode actions are intentionally NOT audited — they're already isolated in the sim event log and don't touch real state.

Failed mutations toggle

`audit.log_failed_mutations` setting (default `False`). When ON, the middleware also writes audit rows for 4xx/5xx mutating responses. Use for defect tracking ("operator hit SUBMIT and saw 422 — what blocked?"); toggle off otherwise to avoid volume spikes from validation errors.

Category routing

`_PATH_CATEGORY_RULES` in `audit.py` is the prefix-match table. Adding a new business surface is one tuple. Example:

```
_PATH_CATEGORY_RULES: tuple[tuple[str, str], ...] = (
    ("/api/orders/ticket",                  "order.place"),
    ("/api/orders/postback",                "order.fill"),
    ...
)
```

For `PUT/PATCH /api/orders/{id}` and `DELETE /api/orders/{id}`, the middleware narrows the generic `order` category to `order.modify` / `order.cancel` based on HTTP method.

UI — filter pills

`/admin/audit` carries a row of category pills (All / Orders / Agents / Users / Config / System) above the existing column filters. Each pill passes a comma-separated category list to the backend; SQL `IN (...)` does the rest. The

Category column in the table carries per-bucket tints (green orders / cyan agents / amber users / violet config / slate system).

Performance contract

- Every audit write is `asyncio.create_task(_write_audit(...))` — no `await`. The middleware's `handle()` returns immediately after the wrapped response.
- The DB insert pays no transaction beyond its own session; failures swallow into a logger warning.
- Helper invocations from background tasks pay the same fire-and-forget cost.
- Read path is one paginated query with `LIMIT/OFFSET`; the `(category, created_at)` and `(actor_user_id, created_at)` indexes cover the common UI queries.

Source files

- `backend/api/audit.py` — middleware + `write_audit_event` helper
- `backend/api/models.py::AuditLog` — table
- `backend/api/routes/audit.py` — read surface + filters
- `frontend/src/routes/(algo)/admin/audit/+page.svelte` — viewer + pills

22.8. Postback fan-out — book_changed bus

Single coordinated refresh trigger for every position-derived surface after a broker postback. Replaces the prior pattern where each surface polled its own cadence and downstream aggregates (Snapshot grid totals, strategy analytics, payoff curve) lagged the per-cell qty patch by 5–15 s.

TECH — Coordinated invalidation vs per-page polling — `WHY` the postback handler historically invalidated only the `orders` cache. `positions` and `holdings` had their own 30 s TTL, and the strategy endpoint memoised its own analytics — so the Snapshot grid showed patched per-cell qty (via `position_filled` optimistic patch) but stale aggregates until the next per-page poll cycle. Operator's report: "snapshot grid updated two iterations." `WHAT` Terminal postbacks now invalidate every dependent cache atomically + broadcast a single `book_changed` event. A frontend singleton subscriber re-emits via a Svelte store; every position-derived page subscribes once and refetches its primary loader. `HOW` 200 ms upstream debounce coalesces basket-order bursts. Monotonic counter store lets `$effect` re-run on every increment without payload comparison. `WHERE backend/api/routes/orders.py::order_postback` + `frontend/src/lib/data/bookChanged.js`.

Backend chain

`POST /api/orders/postback` on any terminal status (COMPLETE / CANCELLED / REJECTED / EXPIRED):

```
invalidate("orders")
if _terminal:
    invalidate("positions")
    invalidate("holdings")
    broadcast({
        "event": "book_changed",
        "account": masked, "exchange": ..., "tradingsymbol": ...,
        "reason": status, "ts": int(time() * 1000),
    })
if status == "COMPLETE":
    broadcast({"event": "position_filled", "qty": signed_delta, ...})
```

`position_filled` is preserved alongside the new event — it carries the qty delta the per-cell optimistic-patch path on Pulse + Performance reads. `book_changed` is the broader coordination signal that also covers CANCELLED / REJECTED paths where there's no qty to patch.

Frontend bus

`$lib/data/bookChanged.js`:

- Singleton subscriber via `createPerformanceSocket`. Started from `(algo)/+layout.svelte::onMount` so every algo page sees it. Idempotent — multiple `startBookChangedBus()` calls share one WS.
- Listens for `book_changed`, debounces 200 ms, increments `bookChanged` (a monotonic counter writable store) + sets `lastBookEvent` (latest payload).
- Pages subscribe with a `$effect` that watches the counter and calls their primary loader once per increment. Counter pattern (vs payload comparison) lets the effect re-run trivially on every change.

Subscription map

Page	Loaders called on increment
`/admin/derivatives`	`loadPositions() + loadStrategy()`
`/dashboard`	`loadHero()`
`/pulse`	`loadPulse()`
`/orders`	`_debouncedLoadOrders()`
`/performance`	`loadAll({ fresh: true })`

Recipe — wire a new page to the bus

```
<script>
  import { bookChanged } from '$lib/data/bookChanged';

  async function loadXxx() { /* page's primary loader */ }

  let _bookCounter = 0;
  $effect(() => {
    const n = $bookChanged;
    if (n <= _bookCounter) return;
    _bookCounter = n;
    loadXxx();
  });
</script>
```

That's it. The counter guard prevents re-entry; the upstream debounce handles burst coalescing.

Performance contract

- Backend fan-out runs inside the postback handler's existing `_asyncio.create_task` block — zero added latency on the broker ack path.
- One extra JSON broadcast per terminal status (~150 bytes wire). At 10 fills/sec that's 1.5 KB/sec across all WS clients combined.
- Frontend debounce keeps loader calls to one per 200 ms window per page.
- Pages without the wiring fall back to their existing pollers — additive, never breaks the prior path.

Source files

- `backend/api/routes/orders.py::order_postback` — invalidation chain + broadcasts
- `frontend/src/lib/data/bookChanged.js` — singleton subscriber + stores
- `frontend/src/routes/(algo)/+layout.svelte` — `startBookChangedBus()` callsite
- Wired pages: admin/derivatives, dashboard, MarketPulse, orders, PerformancePage

---

22.9. History — multi-day orders / trades / funds

`/admin/history` is the row-level forensic surface — three tabs over the platform's append-only datasets: Orders (`algo_orders`), Trades (`daily_book.kind='trades'`), Funds (`daily_book.kind='funds'`). Companion to `/admin/audit` (event-level log); same `view_audit` cap, different storage shape.

TECH — Append-only daily_book vs broker SDK — `WHY` Kite Connect (and most Indian broker SDKs) expose ONLY today's orders + trades; historical data must be scraped from the Console UI. RamboQuant snapshots every loaded account at 15:35 IST into `daily_book` so the platform owns the multi-day record-of-truth without depending on the broker's UI export. `WHAT` One row per (date, account, kind, symbol) with a unique constraint that lets re-runs upsert idempotently. `HOW` `_task_daily_snapshot` background task fires at 15:35 IST + on startup. `WHERE` `backend/api/algo/daily_snapshot.py` + `backend/api/models.py::DailyBook`.

Endpoint surface

`backend/api/routes/history.py::HistoryController` — `view_audit` cap, three reads:

Endpoint	Source	Default range	Pagination
`GET /api/admin/history/orders`	`algo_orders`	30 days	50/page, cap 500
`GET /api/admin/history/trades`	`daily_book[kind='trades']`	30 days	50/page, cap 500
`GET /api/admin/history/funds`	`daily_book[kind='funds']`	90 days	unpaged

Shared params: `from_date / to_date / accounts / symbols` (comma-separated lists for accounts + symbols). Orders adds `status / mode`. Funds drops `symbols`.

Response shape highlights

- **Orders**: `counts` field is a SQL-side `GROUP BY status` histogram. UI renders as summary pills without paginating.
- **Trades**: `summary.total_notional` is `Σ qty × avg_cost` across the FILTERED set, computed via `_func.sum()` so pagination doesn't degrade accuracy.
- **Funds**: `earliest_date` is `MIN(daily_book.date) WHERE kind='funds'` — the UI's "tracking started X" chip uses it to set expectations while historical backfill catches up.

Funds capture (new — Jun 2026)

`_funds_rows` — runs alongside the existing holdings / positions / trades capture inside `_task_daily_snapshot`. Per account, per segment (equity / commodity), one row per day. Idempotent via the existing `daily_book` ON CONFLICT clause.

Column mapping (re-using the generic `daily_book` schema to avoid a new table):

daily_book column	Funds semantic
qty	utilised.debits (□ debited today)
avg_cost	available.cash
ltp	available.opening_balance
day_pnl	utilised.realised_m2m
total_pnl	net (segment net worth)
symbol	'__seg__' sentinel (unique constraint requires non-null)
exchange	segment label uppercased

The mapping is intentionally pragmatic — daily_book is denormalised by design, and adding a separate funds_book table would duplicate the schema without adding value. The semantics are clear from kind='funds'.

Drill, delta, backfill — closed limits

Per-row audit drill — closed. algo_orders.request_id (nullable VARCHAR(36), indexed) captured on POST /api/orders/ticket from request.scope.state.request_id stamped by AuditMiddleware. GET /api/admin/audit accepts a request_id filter param; the audit page reads ?request_id=… URL param on mount + widens since_hours to 90 days. History Orders tab grows an Audit ↗ column per row that opens /admin/audit?request_id=<uuid> pre-filtered.

Cashbook Δ on Funds tab — closed. FundsRow.cash_delta computed server-side: HistoryController.list_funds walks rows in O(N), groups by (account, segment), sorts ASC by date, sets prior_cash to the previous row's cash_available each step. Response keeps DESC order (newest first) for the UI; per-row delta carries the move within the (account, segment) series. UI tints positive green / negative red / em-dash for the first row in a series.

Funds backfill — endpoint + Dhan adapter both wired.

Adapter contract:

```
def funds_ledger(self, from_date: str, to_date: str) -> list[dict]:
    """Return a list of normalised per-(date, segment) rows:
        [{date, segment, cash_available, opening_balance,
          debits, realised_m2m, net, payload}, ...]
    """
```

Endpoint flow:

```
@post("/funds/backfill", guards=[admin_guard])
async def backfill_funds(...) -> FundsBackfillResponse:
    broker = get_broker(account)
    if not hasattr(broker, "funds_ledger"):
        raise HTTPException(status_code=501, detail=...)
    entries = await loop.run_in_executor(
        None, lambda: broker.funds_ledger(from_iso, to_iso))
    # INSERT ... ON CONFLICT DO UPDATE per entry — same column
    # mapping as _funds_rows in the live snapshot path.
```

Broker support matrix:

- **Kite (zerodha_kite)** — no programmatic ledger. Always 501.
- **Dhan** — wired (DhanBroker.funds_ledger). SDK method discovery probes get_ledger_report (v2) /

`get_funds_ledger` / `ledger_report` (fork variants) with kwarg→positional fallback. Aggregates voucher-level entries per `(voucherdate, segment)`; `_DHAN_SEGMENT_MAP` collapses Dhan exchange codes to our 2-segment vocabulary.

- **Groww** — pending. Same single-file pattern: add `funds_ledger(from, to)` to `GrowwBroker` returning the normalised shape.

TECH — Voucher-level aggregation vs daily snapshot

Dhan's `/v2/statement/ledger` returns voucher-level entries (one per transaction: a trade settlement, a brokerage debit, an MTM credit), not daily summaries. The adapter aggregates because `daily_book[kind='funds']` is intentionally per-day per-segment — re-using the existing snapshot schema instead of adding a `funds_ledger_voucher` table.

Aggregation logic: - Group entries by `(voucherdate, segment)`. - Sum `debit` + `credit` separately per group. - Track first + last `runbal` as SOD / EOD proxies (Dhan returns entries in chronological order within a day). - Output `cash_available = close_runbal`, `opening_balance = close_runbal − (credits − debits)`, `realised_m2m = credits − debits` (semantically "net daily cash move", not pure MTM — voucher entries include brokerage / STT / exchange charges that operator should not interpret as P&L).

Idempotency on backfill

The backfill loop uses `INSERT ... ON CONFLICT (date, account, kind, symbol) DO UPDATE SET ...` — same clause as `_upsert_rows` in the daily snapshot path. Re-running a backfill with a wider date range overwrites existing rows with the canonical Dhan numbers (intentional — if both the live snapshot AND a backfill cover the same day, the backfill's voucher-aggregated numbers are more accurate than a single broker.margins() snapshot taken at 15:35 IST).

Per-row try/except + single bulk commit at the end: a single bad voucher entry doesn't lose a multi-month pull. Failed rows log to debug + increment the `skipped` counter; the response surfaces both counts.

Remaining limit

- **Cashbook view as a separate tab** — running-balance walk that reconciles trade-leg deltas against funds snapshots row by row. The Δ column gives the daily move; a dedicated tab could enumerate the trade contributions that produced it. Not in scope for this slice; a follow-up SQL view + 4th tab.

Source files

- `backend/api/routes/history.py` — controller + 3 endpoints
- `backend/api/algo/daily_snapshot.py` — `_funds_rows` + pipeline wiring
- `backend/api/models.py::DailyBook` — table (unchanged; just a new `kind` value)
- `frontend/src/routes/(algo)/admin/history/+page.svelte` — viewer
- `frontend/src/lib/api.js` — `fetchHistoryOrders/Trades/Funds` wrappers

22.10. Order placement latency — preflight + tick cache + paper-skip

Closes the order-placement deterioration the operator flagged ("placement feels slow now"). Three orthogonal fixes ship together because they target the same hot path:

TECH — Sequential awaits vs asyncio.gather — `WHY await` on a `run_in_executor` blocks the route until the broker SDK returns; four sequential awaits = ~800-1200ms on Kite's typical 200-300ms round-trip. Even though

each call is itself async-scheduled, the await chain serializes them. `WHAT` Wrap each independent broker call in its own helper coroutine with self-contained exception handling, then fire all four with `asyncio.gather`. Total wall-time becomes `max(individual call)` instead of `sum(individual calls)`. `HOW` Each helper returns a plain Python value on success or a sentinel (None / tuple-with-error) on failure, so the consumer can branch on result type rather than handle exceptions across the gather boundary. `WHERE backend/api/algo/actions.py::run_preflight` — fan-out helpers `_fetch_profile / _fetch_instruments / _fetch_basket_margin / _fetch_account_margins`.

Fix 1 — preflight parallelization

`run_preflight` previously ran four broker calls in strict sequence:

```
profile = await loop.run_in_executor(None, broker.profile)         # ~300ms
instruments = await loop.run_in_executor(None, broker.instruments, exchange)  # ~250ms
bm_result = await loop.run_in_executor(None, broker.basket_order_margins, …)  # ~300ms
m = await loop.run_in_executor(None, broker.margins)               # ~300ms
# Total: ~1150ms sequential
```

The new structure:

```
# Stage 1 (synchronous) — build basket_orders (uses cached get_lot_size).
# Stage 2 (parallel) — gather the 4 independent broker calls:
profile_res, instruments_res, bm_res, margins_res = await asyncio.gather(
    _fetch_profile(),         # None for non-Kite brokers
    _fetch_instruments(),     # None when exchange not in F&O / qty<=0
    _fetch_basket_margin(),   # Exception on failure (handled below)
    _fetch_account_margins(), # (seg_dict, error_str) tuple
)
# Total: ~max(300ms) parallel
```

Each helper handles its own exceptions so a single broker failure surfaces as a logged warning + None result rather than tearing down the gather. `_fetch_basket_margin` returns the exception object (not raising) so the consumer can re-raise into the existing MARGIN_SHORTFALL block's try/except — minimal change to the downstream handler.

Fix 2 — tick-size index

`_align_price_to_tick` looked up the contract's tick_size via a linear scan:

```
for inst in items:              # items = 10–50k rows
    if inst.s == sym_u and inst.e == ex_u:
        tick = float(inst.ts or 0)
        break
```

Ticket route called this twice per order (price + trigger), so a single ticket paid ~100k linear iterations.

Now a module-level `_TICK_INDEX: dict[tuple[str,str], float]` is built lazily from the instruments cache. `_TICK_INDEX_STAMP` holds the cached `InstrumentsResponse` object; identity comparison (`resp is not _TICK_INDEX_STAMP`) detects cache refresh and triggers a rebuild. Subsequent ticket calls are O(1) dict lookups.

Trade-off: the rebuild itself is still O(N) — one scan per cache refresh (typical TTL ~10 minutes). Hot-path savings dominate; ~50ms per ticket recovered.

Fix 3 — PAPER skips route-level preflight

`PaperTradeEngine.register_open_order` already runs `basket_order_margins` internally — it's the gate that decides REJECTED vs OPEN on an open order, and writes the broker's exact error string into `AlgoOrder.detail` when the basket margin check fails. The route-level preflight before that was running the SAME basket_margin call (plus three others) for the SAME order, costing ~800ms with zero additional correctness.

The PAPER branch of `ticket_order` no longer calls `run_preflight()`. LIVE preflight stays — it's the only chance to block before `kite.place_order` actually fires.

Combined ticket-path savings

Path	Before	After
LIVE ticket	~1200ms preflight + ~150ms route + ~300ms place_order ≈ 1.65s	~300ms preflight + ~150ms route + ~300ms place_order ≈ 0.75s
PAPER ticket	~1200ms preflight + ~150ms route + ~50ms engine register ≈ 1.40s	~150ms route + ~50ms engine register ≈ 0.20s

PAPER is the bigger win because the entire preflight goes away; LIVE saves roughly half the latency.

Source files

- `backend/api/algo/actions.py::run_preflight` — parallel gather
- `backend/api/routes/orders.py::_align_price_to_tick` + `_TICK_INDEX` — O(1) tick lookup
- `backend/api/routes/orders.py::ticket_order` — PAPER preflight skip block

22.11. Navbar audit — rename + resequence

Operator-requested audit of the algo navbar.

Renames:

- `modes` group → `explore`. The old name was vestigial from the sim/paper/live/shadow/replay terminology before the mode toggles moved to the navbar dropdown (Wave C). Group now contains just `Sandbox`; can grow when Replay gets its own dedicated entry.
- `Lab` label → `Sandbox`. Industry-standard term across QuantConnect / Streak / Sensibull; reads faster to first-time visitors than the prior internal jargon. URL `/admin/execution` unchanged — bookmarks + deep links preserved.

Monitor resequence (rationale = daily-trader workflow frequency):

```
old: Tour Pulse Dashboard Derivatives Strategies NAV Orders Charts Automation
new: Tour Pulse Dashboard Orders        Derivatives Charts Automation Strategies NAV
```

Orders moved ahead of analysis surfaces (Derivatives / Charts) since active trading is the trader's primary entry point. Strategies + NAV move to the end — attribution + LP-facing views are weekly, not minute-by-minute.

Implementation:

```
// frontend/src/routes/(algo)/+layout.svelte
const GROUP_LABELS = {
  monitor: 'Monitor',
  analyze: 'Analyze',
```

```
  explore: 'Explore',
  build:   'Build',
  config:  'Config',
};
const INLINE_GROUPS = new Set(['monitor', 'analyze', 'explore']);
```

`INLINE_GROUPS` controls which groups render inline in the desktop nav; the rest collapse to dropdown triggers. Mobile drawer shows every group with a `GROUP_LABELS` caption for scan-by-intent navigation.

22.12. #audit workflow + Dhan / Groww postback scaffold

The operator runs periodic comprehensive audits by writing the literal hashtag `#audit` in chat. The coding agent dispatches 8 parallel `audit` subagents (one per dimension: performance / defects / stale code / UX consistency / palette / broker-API parity / data layer / docs) and synthesizes findings into a severity-tagged punch list. Audit findings ship as `audit slice <letter>` commits.

TECH — Parallel audit subagents vs single deep review — `WHY` A single agent asked to cover 8 dimensions produces shallow work on each. Eight focused subagents each get a tight scope brief, run concurrently, and report independently. `WHAT` 8 Agent tool-calls in one message with `subagent_type=audit` (read-only), each prompt cites the canonical patterns to check against. `HOW` Each subagent returns a focused punch list with severity tags (HIGH / MED / LOW); the coordinator synthesizes into a remediation plan rather than firing fixes unilaterally. `WHERE` Memory file `~/.claude/projects/-Users-ramanambore-projects-ramboq/memory/feedback_audit_tag.md` documents the workflow.

Audit slices A, B, C (shipped Jun 2026)

Slice A — quick wins: doc drift in CLAUDE.md + README.md (navbar renames `modes`→`explore`, `Lab`→`Sandbox`, monitor sequence); stale code purges (`OrderDetail.svelte`, `margin_optimizer.py`, `shadow.py` + `ShadowController`, 4 api.js shadow stubs, `fetchPnlRange` — all confirmed unreferenced); palette fixes (LogPanel border `#10b981`→`#4ade80`; RefreshButton badges 600-level→400-level).

Slice B — perf + data layer: - `_task_performance` ticker subscribe loop, `_task_trail_stop`, `_task_oco_pair_watcher`: sequential per-account awaits → `asyncio.gather`. Each saves ~200-300ms × N accounts. - `get_lot_size` O(N) → O(1) via `_LOT_INDEX` dict (same identity-stamp invalidation pattern as `routes/orders.py::_TICK_INDEX`). - 3 DB indexes added in `init_db` migration block (idempotent `CREATE INDEX IF NOT EXISTS`): - `ix_algo_orders_trail_stop` — partial on (mode, status) WHERE attached_gtts_json IS NOT NULL - `ix_news_headlines_published_at` — DESC - `ix_strategy_lots_open` — composite missing from create_all on pre-existing tables - Operator-visible interrupt fixes batched in: - `/admin/history` + `/admin/audit`: $effect-gated load (was `onMount` checking `_canView` once at false; load never fired) - Algo layout: removed blanket "non-admin → /signin" redirect (vestigial from old admin/partner tier; broke tour for trader/risk/admin roles — pre-rename names were ops/observer) - "Prev Close" → "Close" rename across PerformancePage, MarketPulse, /admin/derivatives

Slice C — defects + Dhan/Groww postback scaffold: - `list_active_chases` template-attach gap: live-reconcile path flipped FILLED but never called `_maybe_fire_template_attach_for_reconcile`. Now captures `_reconciled_filled` rows + fires after commit. - `chase.py:806` partial-fill slippage formula: `quantity` → `filled_qty or quantity` (the old formula overstated slippage when partials happened). - New routes `POST /api/orders/{dhan,groww}_postback` with shared `_process_broker_postback` helper that mirrors the Kite path's fan-out (AlgoOrder sync + audit log + cache invalidate + WS broadcasts). Best-effort; logs raw payload on first

hit so parser can be tuned.

`_process_broker_postback` shared helper

`backend/api/routes/orders.py::_process_broker_postback` — extracted from the Kite postback inline logic so Dhan + Groww routes call the same fan-out:

```
async def _process_broker_postback(
    *, broker_id, order_id, status, account, symbol, txn, qty, price,
    exchange="", status_message="",
):
    # 1. AlgoOrder row sync by broker_order_id (status, fill_price, filled_at)
    # 2. order_events row (broker_postback kind)
    # 3. audit_log entry tagged order.fill|cancel|reject
    # 4. invalidate orders / positions / holdings on terminal
    # 5. broadcast order_update + position_filled + book_changed
```

Status normalization uses `_DHAN_STATUS_TO_KITE` and `_GROWW_STATUS_TO_KITE` tables already in the broker adapters. Best-effort throughout; never 5xx so the broker doesn't retry.

Source files

- `backend/api/algo/actions.py::run_preflight` — parallel gather (slice 22.10)
- `backend/api/background.py::_task_{performance,trail_stop,oco_pair_watcher}` — slice B parallel gatherers
- `backend/brokers/adapters/kite.py::get_lot_size` + `_LOT_INDEX` — slice B O(1) lookup
- `backend/api/database.py::init_db` — slice B index migrations
- `backend/api/routes/orders.py::list_active_chases` — slice C template-attach fix
- `backend/api/algo/chase.py:806` — slice C slippage formula
- `backend/api/routes/orders.py::order_postback_{dhan,groww}` + `_process_broker_postback` — slice C postback scaffolds

22.13. Audit slice D — UX consistency + palette consolidation + 2 defects

Closes the remaining MED-severity items from the #audit run. Pattern: converge on canonical components + canonical CSS-custom-prop alphas rather than introducing new code.

UX consistency — adopting canonical components

Three pages were running their own bespoke implementations of patterns the codebase already had canonical components for. Each replacement deletes the bespoke chrome + the CSS that supported it:

Page	Bespoke pattern	Canonical replacement
`/admin/history`	`.hist-tabs` + `.hist-tab` (+ active border + count badge)	`<AlgoTabs tabs={[{id, label, badge}, ...]} bind:value={tab} onChange={setTab} />`
`/admin/statements`	native `<select class="ms-select">` + `.ms-select` CSS block	`<Select bind:value={selectedPeriod} options={_months} />`
`PageHeaderActions` amber+cyan hover bg	`rgba(_, 0.12)` (drift)	`rgba(_, 0.14)` matching `--algo-amber-bg` / `--algo-cyan-bg` canonical

 TECH — Canonical-component adoption vs maintaining bespoke chrome — WHY Every bespoke tab strip / select / pill the operator can't visually distinguish from the canonical ones is friction on muscle memory. Slice D's audit found 4 parallel pill-strip implementations across audit / statements / templates / agent-templates pages — same job, four different CSS classes. WHAT Replace bespoke implementations with the existing canonical component when the contract matches; the canonical component does the styling once and every page inherits. HOW `AlgoTabs` already exposes `tabs: [{id, label, badge, color}]` so the History tab badge logic (showing total only on the active tab) maps trivially. `Select` accepts `options: [{value, label}]` so the period dropdown's already-correct data shape is a one-line swap. WHERE `frontend/src/lib/AlgoTabs.svelte`, `frontend/src/lib/Select.svelte`.

Palette alpha consolidation

The cyan-bg alpha `0.10` was drifting across 5 files (8 callsites) while the canonical `--algo-cyan-bg` is `0.14`. CommandBar / HireMeModal / OrderCard / OrderTicket / SymbolPanel all converged. Same fix on PageHeaderActions amber `0.12` → `0.14`.

Net effect: a chip background on one page now visually matches the same conceptual element on another page. Pre-fix the operator's brain had to disambiguate "is this cyan-12 or cyan-14?" — now both reach `var(--algo-cyan-bg)`.

Defects

`paper.py::reset()` race — pre-fix the three dict-replace operations (`_open_orders`, `_price_history`, `_underlying_history`) ran unlocked. If a concurrent `step()` snapshot or `_capture_price_history` write happened during the replace, the price-history chart data for the first tick of a new sim could silently land in the OLD dict reference while the new sim queried the empty replacement. Fix: acquire `self._lock` around the replacements.

`history.py::backfill_funds` docstring — was claiming "idempotent — existing rows are not overwritten (ON CONFLICT DO NOTHING)" but the SQL is `DO UPDATE SET ...` (full overwrite). Operator-surprise risk: a hand-edit to a funds row gets clobbered on the next backfill with the same date range. Docstring now accurately documents the overwrite + warns the operator to treat funds rows as read-only.

Source files

- `frontend/src/routes/(algo)/admin/history/+page.svelte` — `<AlgoTabs>` adoption
- `frontend/src/routes/(algo)/admin/statements/+page.svelte` — `<Select>` adoption
- `frontend/src/lib/PageHeaderActions.svelte` — amber/cyan 0.12→0.14
- `frontend/src/lib/{CommandBar,HireMeModal,SymbolPanel}.svelte` + `order/{OrderCard,OrderTicket}.svelte` — cyan 0.10→0.14
- `backend/api/algo/paper.py::reset` — `self._lock` acquisition
- `backend/api/routes/history.py::backfill_funds` — docstring correction

22.14. Market-status — broker API beats bellwether-quote probe

The agent engine's `market_hours` schedule gate and the daily snapshot pipeline both consult `probe_market_active(exchange)` to decide whether the market is currently trading. Pre-slice-E the probe used a workaround: call `kite.quote()` on bellwether symbols (NIFTY 50 + NIFTY BANK for NSE/NFO, SENSEX for BSE/BFO, or the dynamically-resolved nearest MCX commodity futures contract — not hardcoded CRUDEOIL), check `last_trade_time` freshness within a 15-minute window. Worked, but spent Kite's quote budget on a question Kite's API can't answer directly.

 TECH — Authoritative broker API vs inferred bellwether probe — WHY Kite Connect has no market-status endpoint; the only signal Kite exposes is a quote with `last_trade_time`. Dhan and Groww both ship a direct market-status API (`get_market_status` and variants). When an authoritative answer is one round-trip away, prefer it over an inferred one — bellwether probes have edge cases (illiquid contracts, weekend Muhurat sessions, MCX evening sessions) where the inference disagrees with the broker. WHAT Extend the `Broker` ABC with an optional `market_status(exchange) -> bool | None` method. Adapters that have the API override and return True/False. The probe layer iterates brokers, takes the first definitive answer, falls back to the bellwether path when no broker answers. HOW SDK-method discovery probes (`getattr(sdk, 'get_market_status', None)` etc.) handle adapter version drift. Per-exchange 60s cache absorbs the per-tick gate evaluation. WHERE `backend/brokers/base.py::market_status`, `backend/brokers/adapters/dhan.py::market_status`, `backend/brokers/adapters/groww.py::market_status`, `backend/shared/helpers/market_probe.py::probe_market_active`.

Resolution order

```
probe_market_active(exchange):
  if cache hit and fresh (60s TTL): return cached
  for broker in all_brokers():
    verdict = broker.market_status(exchange)   # ← step 1
    if isinstance(verdict, bool):
      cache[exchange] = verdict
      return verdict
  # step 2: fall back to bellwether-quote probe (unchanged path)
  kite = resolve_kite_handle()
  if kite is None: return None
  bellwethers = _candidates(exchange, broker)
  q = kite.quote(bellwethers)
  active = any(row.last_trade_time >= now - 15min for row in q)
  cache[exchange] = active
  return active
```

Adapter contract

```python
class Broker(ABC):
    def market_status(self, exchange: str) -> bool | None:
        """True / False if broker exposes a market-status endpoint
        for `exchange`; None when adapter doesn't implement one or
        the call fails. Optional method, not abstract."""
        return None
```

Both Dhan and Groww implementations probe known SDK method names (`get_market_status` / `market_status` / `get_exchange_status`) across SDK version drift; iterate the response rows (whatever shape the SDK returns); map our exchange vocabulary (NSE / BSE / NFO / BFO / CDS / MCX) to the broker's segment codes (Dhan: `NSE_EQ` / `BSE_EQ` / `NSE_FNO` / `BSE_FNO` / `NSE_CURRENCY` / `MCX_COMM`; Groww: similar with variants); return `True` if ANY mapped segment reports active.

Side effects on quote budget

A typical Kite-only deployment hits the bellwether path on every cache miss 4 symbols × 1 quote call ≈ 4 instruments off the 10-req/sec quote budget per probe. A Dhan-loaded deployment skips that entirely for any

exchange Dhan covers; only the rare cache-miss-with-Dhan-down case falls through.

Adding the API to a new adapter

```
def market_status(self, exchange: str) -> bool | None:
    sdk = self.client
    status_fn = (getattr(sdk, "get_market_status", None)
                 or getattr(sdk, "market_status", None))
    if status_fn is None:
        return None
    try:
        resp = self._safe_call(lambda c: status_fn())
    except Exception as e:
        logger.debug(f"{self.broker_id}.market_status failed: {e}")
        return None
    # map your broker's segment codes → our vocabulary
    # return True if any mapped segment reports active
    # return False if all closed
    # return None if no mapping matched (fall through to next broker)
```

Source files

- `backend/brokers/base.py::market_status` — ABC default
- `backend/brokers/adapters/dhan.py::market_status` — Dhan implementation
- `backend/brokers/adapters/groww.py::market_status` — Groww implementation
- `backend/shared/helpers/market_probe.py::probe_market_active` — resolution chain + cache

22.15. Chart indicator system — pure module + overlay persistence

Technical indicators (SMA, EMA, VWAP, Bollinger Bands, RSI, MACD) live in a single pure module rather than being inlined in ChartWorkspace.

TECH — Indicators as a pure stateless module — `WHY` Inline math inside a 2000-line Svelte component is untestable with `node --test` (no DOM, no Svelte runtime needed). Extracting the math to a pure module means a 32-test suite can verify hand-calculated reference values, edge cases (empty arrays, N=0, constant series), and Wilder-smoothing correctness without a browser. `WHAT` `frontend/src/lib/chart/indicators.js` exports `sma`, `ema`, `vwap`, `bollinger`, `rsi`, `macd`. Each function takes an OHLCV bars array, returns a typed series array. First (n-1) entries are `{ts, value: null}` — warmup convention. `HOW` `_assertN(n)` throws `RangeError` for non-positive or non-integer periods. MACD throws when `fast >= slow`. All functions are pure (no side-effects, no imports). `WHERE` `frontend/src/lib/chart/indicators.js`; tests at `frontend/scripts/indicators.test.js`.

Overlay palette (canonical colours, do not vary):

Overlay	CSS class	Colour	Notes
SMA 20	`overlay-sma`	`#7dd3fc` sky-blue	dashed 4-3
SMA 50	`overlay-sma`	`#c084fc` violet	dashed 6-3
EMA 20	`overlay-ema`	`#4ade80` green	solid
EMA 50	`overlay-ema`	`#fb923c` orange	dashed 6-3
VWAP	`overlay-vwap`	`#7dd3fc` cyan	solid 1.4px

Overlay	CSS class	Colour	Notes
BB mid	`overlay-bb`	`#7dd3fc` cyan	solid 1px
BB upper/lower	`overlay-bb`	`#7dd3fc` cyan	dashed 3-2
BB fill	`overlay-bb`	`rgba(125,211,252,0.06)`	no stroke
RSI line	`overlay-rsi`	`#fbbf24` amber	solid 1.5px
MACD line	`overlay-macd`	`#fbbf24` amber	solid 1.4px
MACD signal	`overlay-macd`	`#f87171` red	dashed 3-2
MACD histogram	(line elements)	`rgba(74,222,128,0.55)` / `rgba(248,113,113,0.55)`	green above zero, red below

Sub-panel geometry (SVG user-unit constants in ChartWorkspace.svelte): - `RSI_H = 48` — RSI panel height - `MACD_H = 56` — MACD panel height - `_bandH = (_showRsi ? RSI_H : 0) + (_showMacd ? MACD_H : 0)` — reserved bottom space - `_innerH = chartH − CPAD_T − CPAD_B − _bandH` — usable height for the price panel

Overlay persistence: - LocalStorage key: `rbq.cache.chart-overlays.v1` (JSON array of string keys) - Init: `$state([])` — empty server-side. Never `$state(_loadPrefs())` because `$state()` init runs during SSR where `localStorage` is undefined. - Hydration: `onMount` reads and validates the stored array against `_OVERLAY_OPTS`. Sets `_overlaysHydrated = true` after reading. - Save: `$effect` watches `_overlays` but guards with `if (!_overlaysHydrated) return` to prevent overwriting stored prefs during the first render frame.

VWAP note — indices (NIFTY 50, NIFTY BANK etc.) carry `volume=0` on every bar. `calcVwap()` returns `null` for all points when `cumVol=0`. The `{#if _vwapPath}` block in the SVG template silently suppresses the element. This is correct: VWAP is a price/volume metric that has no meaning for non-tradeable indices.

Buy / sell signal markers

 TECH — Signal detection as pure helpers, render layer pure SVG — `WHY` Operator brief: surface buy/sell points TradingView-style for each active indicator. Detection logic must be testable (`node --test`) and the marker layer must respect the canonical algo palette so the visual vocabulary matches the rest of the app. `WHAT` Five exports in `indicators.js` — `emaSignals(fast, slow)`, `vwapSignals(closes, vwapArr)`, `bollingerSignals(closes, bb)`, `rsiSignals(arr, oversold=30, overbought=70)`, `macdSignals(macdLine, signalLine)`. Each returns `[{i, type:'buy'|'sell'}]`. Inputs are duck-typed — they accept raw number arrays, `{value}` arrays (real ema output), or `{close}` bars (raw OHLCV). `HOW` In ChartWorkspace, `_signalMarkers` is a `$derived.by` that re-runs only when `_bars` or `_overlays` change; `_signalLayout` translates events to `{x, y, type, tag, tooltip, stack}` records with same-bar stacking. SVG renders one `<g class="signal-marker signal-{type}">` per event with a triangle + 9px monospace tag. `WHERE` `frontend/src/lib/chart/indicators.js::emaSignals|vwapSignals|bollingerSignals|rsiSignals|macdSignals`; `frontend/src/lib/ChartWorkspace.svelte::_signalMarkers|_signalLayout`.

Marker palette + geometry (canonical, do not vary):

Element	Spec
Buy triangle	filled `#4ade80` emerald-400, 10×8 px, anchored at bar's low + 8 px pad, tip-up
Sell triangle	filled `#f87171` red-400, 10×8 px, anchored at bar's high − 8 px pad, tip-down
Stroke	`#0a0a0a` 0.5 px (subtle outline so triangles read against the chart background tint)
Tag font	9 px monospace, weight 700, paint-order stroke-then-fill with 2.5 px dark stroke for legibility against bars
Tag colour	matches triangle (`#4ade80` buy, `#f87171` sell)

Element	Spec
Stack offset	16 px vertical between markers on the same bar (split: buys below, sells above)
Indicator tag text	`EMA↑` / `EMA↓` / `VWAP↑` / `VWAP↓` / `BB↓` (buy = lower band) / `BB↑` (sell = upper band) / `RSI↑` / `RSI↓` / `MACD↑` / `MACD↓`
Tooltip (`<title>` element)	`Buy signal – RSI 14 @ 2026-04-15` (verb + indicator + bar timestamp)
Density throttle	per-indicator cap of 12 events on dense ranges (`_bars.length >= 180`); most-recent events kept
Bollinger throttle	first bar of a contiguous lower / upper band run only — prevents 5-marker spam on multi-bar breaks

Signal detection rules (peer-platform standard — TradingView / Sensibull / Streak / Upstox):

Indicator	Buy	Sell
EMA cross	fast > slow AND prev fast ≤ prev slow (golden cross)	fast < slow AND prev fast ≥ prev slow (death cross)
VWAP	close > vwap AND prev close ≤ prev vwap	close < vwap AND prev close ≥ prev vwap
Bollinger	close ≤ lower band (first bar)	close ≥ upper band (first bar)
RSI 14	rsi > 30 AND prev rsi ≤ 30	rsi < 70 AND prev rsi ≥ 70
MACD 12/26/9	macd > signal AND prev macd ≤ prev signal	macd < signal AND prev macd ≥ prev signal

Toggle UX — Signals chip in chart toolbar renders only when `_overlays.length > 0` (no markers to show without an indicator). Default ON, persisted to `localStorage` key `rbq.cache.chart-signals.v1`. Same height + active-state palette (cyan-400) as the Intraday chip — toolbar height SSOT (`--chart-toolbar-h`) preserved.

Source files

- `frontend/src/lib/chart/indicators.js` — pure indicator + signal functions
- `frontend/src/lib/ChartWorkspace.svelte` — imports `calcEma`, `calcVwap`, `calcMacd`, `calcBollinger`, `calcRsi`, + 5 signal helpers; all overlay paths and `_signalMarkers` / `_signalLayout` are `$derived`
- `frontend/scripts/indicators.test.js` — 52-test unit suite (`node --test`) covering indicator math + 5 signal helpers
- `frontend/e2e/chart_overlays.spec.js` — indicator paths Playwright spec
- `frontend/e2e/chart_signals.spec.js` — buy/sell markers Playwright spec (chromium-desktop + mobile-portrait)

Part VII — Operations

23. How to add a new template field

Templates have grown organically. The current schema is wide (5 mandatory + 7 optional fields). To add a new one:

Backend

1. **Add the column** to `OrderTemplate` in `backend/api/models.py`.

2. **Idempotent ALTER TABLE** in `backend/api/database.py::init_db`.
3. **Schema fields** in `backend/api/schemas.py` — `OrderTemplate` (response), `OrderTemplateCreate`, `OrderTemplatePatch`. Also `TicketOrderRequest` + `BasketLeg` if you want a per-submit override.
4. **`_build_overrides_json`** in `orders.py` (search for the function name) — add the override JSON key.
5. **`resolve_template_plan`** in `template_attach.py` — add the `_pick()` call and the GTT spec emission.
6. **Seeded defaults** — update `SYSTEM_TEMPLATES` in `templates_seed.py` if your field should ship with a value.

Frontend

7. **Template management UI** at `/automation/templates` — add the input.
8. **Override input** at the shell-level Template container in `SymbolPanel.svelte` — add the override field + reset on template change.
9. **Preview** — `previewTicketTemplate` should already wire it because the backend handles it; double-check the chip render handles the new shape.

Documents

10. **Add a row** to §10 if it's a default field.
11. **Update §11** if your field has unusual merge semantics.

---

24. Testing philosophy

The codebase has fewer tests than ideal — that's a known debt. Where tests exist:

- **`backend/tests/`** — pytest + pytest-asyncio. Run via `pytest backend/tests/`.
- **`frontend/e2e/`** — Playwright. Run via `cd frontend && npx playwright test`.
- **No unit tests for frontend** — relies on `svelte-check` + manual flows + e2e.

The Playwright tests run against `dev.ramboq.com` (deployed dev branch). They're slow but high-confidence. Use them for any UX flow that changes; backend pytest for any algo/broker change.

Rule of thumb: if you're touching `chase.py`, `template_attach.py`, or any broker adapter, add a pytest test. If you're touching SymbolPanel / OrderTicket flow, add a Playwright spec.

 TECH — Why Playwright over Cypress — `WHY` Multi-tab support, native browser context isolation, better async waits. Cypress's same-origin restrictions don't fit our auth flow (OAuth-like JWT). `WHAT` Specs in `frontend/e2e/*.spec.js`. Run with `--workers=1` so dev DB writes don't race. `HOW` Use `expect(...).toContainText(...)` for chip assertions; `toHaveAttribute('placeholder', ...)` for input placeholders. `WHERE` `frontend/e2e/`.

---

25. Logging discipline

Three log files matter:

- `api_log_file` — full API log (5MB rotating × 5). Read this first when debugging.
- `api_error_file` — stdout+stderr tee from systemd. Catches uncaught exceptions.
- `hook.log` — webhook listener output.

Log levels by intent: - `DEBUG` — for trace-style detail. Verbose; filtered out in prod. - `INFO` — operator-visible events. Order placed, agent fired, chase replaced. - `WARNING` — recoverable failures. Broker auth retry, asymmetric GTT, partial OCO failure. - `ERROR` — uncaught exceptions, lost state. Should also trigger Telegram.

Don't log inside hot loops without a rate limit. `_task_performance` ran a `logger.info` per row early on; quickly buried `api_log_file` under non-actionable noise.

---

26. Deployment notes

Both `dev` and `main` deploy via webhook. Push triggers:

```
GitHub push → webhook.ramboq.com → /etc/webhook/dispatch.sh
  → main:  /opt/ramboq/webhook/deploy.sh prod main
  → other: /opt/ramboq_dev/webhook/deploy.sh dev <branch>
```

`deploy.sh` (per env): 1. `git pull` 2. `pip install` (production deps) 3. `npm run build` (vite) 4. `systemctl restart ramboq_api.service` / `ramboq_dev_api.service` 5. `notify_deploy.py` (Telegram-only since May 2026)

Per-environment serialisation: a host-wide `/tmp/ramboq_deploy.lock` prevents concurrent prod + dev builds from race-condition npm conflicts. `nice -n 19 ionice -c 3` on npm so background builds never starve API responsiveness.

Manual server work after SSH: always `chown -R www-data:www-data /opt/ramboq /opt/ramboq_dev`. Webhook deploys fail silently if file owner is wrong.

 TECH — Webhook-based deploy vs CI/CD platform — `WHY` We're a single-server setup; GitHub Actions would add 30-60s to every deploy plus a $/runner cost. The webhook is bash + git, zero dependencies. `WHAT` `webhook` (Adnan Hajdarbegovic's daemon) listens on port 9000, validates the HMAC, runs `dispatch.sh`. `HOW` Push to a watched branch triggers it automatically. Logs in `hook.log`. `WHERE` `/etc/webhook/hooks.json` (on server); `webhook/dispatch.sh` + `webhook/deploy.sh` (in repo).

---

27. Sprint history + audit fixes

Previous fixes are documented in-code via comments. Key milestones:

Phase/Sprint	Key fixes	Lookup
Phase 0–3	Template attach pipeline (resolve plan GTT place)	grep `Phase \d`
Sprint A–E	Reconcile paths, partial fills, Dhan/Groww OCO, rate limits	grep `Sprint [A-E]`
Gap closure (B–L)	28 audit fixes across categories	`git log --grep="audit fix" -i`

See commit bodies for specific gap IDs (e.g. B-1 = Dhan status map, C-3 = postback fallback window, H-5 = cap warnings). These are documented in code as defensive comments.

Part VIII — Wrap-up

28. Reading order for a new developer

If you've got a week to onboard:

Day 1 — understand the shape: - This doc end-to-end - `CLAUDE.md` skim (it's the operator-facing manual; some route URLs may reference `/agents/*` which has been redirected to `/automation/*` — see §29) - `backend/api/app.py` startup wiring - `backend/api/models.py` schema

Day 2 — order flow: - `frontend/src/lib/SymbolPanel.svelte` + `OrderTicket.svelte` (the modal) - `backend/api/routes/orders.py::ticket_order` (single submit path) - `backend/api/algo/chase.py::chase_order` (the loop)

Day 3 — templates: - `backend/api/algo/template_attach.py` (resolve + apply) - `backend/api/algo/templates_seed.py` (the matrix) - Trace one BUY CE order from click → fill → attach end-to-end

Day 4 — brokers: - `backend/brokers/base.py` (the ABC) - `backend/brokers/adapters/kite.py` (reference impl) - `backend/brokers/adapters/dhan.py` + `groww.py` (vendor quirks)

Day 5 — background + extras: - `backend/api/background.py` (every task) - `backend/api/algo/actions.py` (agent action handlers) - `frontend/src/lib/order/ChaseCard.svelte` + `OrderCard.svelte` (display)

If you've got a day: read §7 (chase loop) above, then read `chase.py::chase_order` source. Everything else extends from that one function.

---

29. When in doubt

Open an `Agent` with `subagent_type=audit` and ask it to trace your specific scenario. The audit agents in this codebase are well-calibrated for finding subtle issues. Don't merge a change to `chase.py` or `template_attach.py` without one.

Known doc-drift in CLAUDE.md (as of the most recent doc audit): the older operator manual still references `/agents`, `/agents/activity`, `/agents/fragments` URLs. These have been redirected to `/automation`, `/automation/activity`, etc. The redirect routes still work; the URLs in CLAUDE.md are just stale. The current canonical URLs are under `/automation/*`.

---

30. Operator's mental model — the one-page summary

Action	Read this section
"What happens when I click Submit on Ticket?"	§5 — single ticket sequence
"What does the chase loop do between attempts?"	§7 — chase lifecycle
"How does TP/SL get attached?"	§9 — template attach pipeline
"Why is my SL not ratcheting on Dhan?"	§13 — trail-stop subsystem
"How does the Default pill pick the right template?"	§10 — 4-default matrix
"When does the preview chip swap on Chain?"	§17 — frontend modal state
"What runs in the background?"	§20 — task topology
"Why does the navbar strip not match the dashboard?"	§21 — data refresh paths
"What can a demo visitor do?"	§22 — demo mode flow

Action	Read this section
“How do I add a new broker?”	§15
“How do I add a new template field?”	§23
“What’s the tech stack?”	§2 — overview; also inline TECH callouts throughout

---

Part IX — Change recipes (cookbook)

This section turns the design knowledge above into runnable change recipes. Each recipe lists the **exact files to edit**, the **exact pattern to copy**, and the **verification step** before commit. Use these as templates — copy-paste, rename, tweak.

The cookbook is intentionally prescriptive. You do not need to read the full doc above to follow a recipe; you only need §3 (architectural principles) for the philosophy, then jump straight here.

---

31. Recipe: add a new route

Scenario: you want a new endpoint, e.g. `GET /api/positions/heatmap` that returns aggregated per-symbol stats.

Steps

1. **Pick the right route file.** `backend/api/routes/` is grouped by domain (`orders.py`, `positions.py`, `holdings.py`, `agents.py`, etc.). Add the new route to the matching file. New domain? Create `heatmap.py`.
2. **Write the msgspec response type** in `backend/api/schemas.py`:

```python
class HeatmapRow(msgspec.Struct):
    symbol: str
    pnl: float
    weight: float
class HeatmapResponse(msgspec.Struct):
    rows: list[HeatmapRow]
    refreshed_at: str
```

3. **Add the route** to the controller (mirror an existing simple route as a template — e.g. `PositionsController.list_positions` in `backend/api/routes/positions.py`):

```python
@get("/heatmap", guards=[auth_or_demo_guard])
async def heatmap(self, request: Request) -> HeatmapResponse:
    is_demo = request.state.is_demo
    rows = await self._build_heatmap()
    if is_demo:
        rows = [_mask_row(r) for r in rows]
    return HeatmapResponse(rows=rows, refreshed_at=timestamp_display())
```

4. **Register the controller** in `backend/api/app.py` if the file is new. Existing controllers don’t need re-registration.

5. **Frontend wrapper** in `frontend/src/lib/api.js`:

```
export const fetchHeatmap = () => _get('/api/positions/heatmap');
```

   The wrapper handles auth, retries, demo masking display, and error trimming automatically. **Never call `fetch()` directly from a component** — always go through `api.js`.

6. **Demo masking.** Read paths must mask account values for demo sessions (§22). Use `mask_column(col)` helper, never roll your own.
7. **Verify.** `pytest backend/tests/test_routes_smoke.py` (add a smoke test if the route is non-trivial) + manual curl.

---

32. Recipe: add a column to an existing table

Scenario: you want `algo_orders.last_chase_quote` to store the last broker depth snapshot.

Steps

1. **Edit `backend/api/models.py`.** Add the field to the SQLAlchemy model:

```
last_chase_quote: Mapped[str | None] = mapped_column(Text, nullable=True)
```

2. **Add an idempotent ALTER** to `backend/api/database.py::init_db`:

```
await conn.execute(text(
    "ALTER TABLE algo_orders ADD COLUMN IF NOT EXISTS last_chase_quote TEXT"
))
```

   The `IF NOT EXISTS` is non-negotiable — `init_db` runs on every startup, idempotency required.

3. **Update msgspec schema** in `backend/api/schemas.py` if the column should be returned over the wire:

```
class AlgoOrderInfo(msgspec.Struct, kw_only=True):
    ...
    last_chase_quote: str | None = None
```

   `kw_only=True` is non-negotiable — the struct interleaves required + optional fields, and Python 3.13's stricter msgspec refuses that without it. Don't strip the modifier when copy-pasting.

4. **Populate it.** Decide which code path writes to it. For our example, `chase.py::chase_order` writes after each depth quote fetch.
5. **Frontend render** (optional). Add a column to `OrderCard.svelte` or `OrderTab.svelte` ag-Grid config; render via cellRenderer.
6. **Verify.** `psql -d ramboq_dev -c "\d algo_orders"` shows the new column; `pytest`; e2e if frontend-visible.

 Never write a migration script. The `init_db ALTER TABLE ... IF NOT EXISTS` pattern is the migration mechanism. We don't use Alembic.

---

33. Recipe: add a new background task

Scenario: you want `_task_unrealized_pnl_alert` that fires every 60s during market hours.

Steps

1. **Define the coroutine** in `backend/api/background.py`. Copy the shape of `_task_oco_pair_watcher` — it's the simplest template:

```
async def _task_unrealized_pnl_alert():
    interval = 60
    while True:
        try:
            await _run_unrealized_pnl_check()
        except Exception as e:
            logger.exception(f"_task_unrealized_pnl_alert failed: {e}")
        await asyncio.sleep(interval)
```

   The try/except around the loop body is non-negotiable — without it, an uncaught exception silently kills the task forever.

2. **Spawn it at startup.** In `backend/api/app.py::on_startup`:

```
asyncio.create_task(_task_unrealized_pnl_alert())
```

3. **Gate by market hours** if appropriate. Use `is_any_segment_open()` from `backend/shared/helpers/date_time_utils.py`:

```
if not is_any_segment_open():
    await asyncio.sleep(interval)
    continue
```

4. **Gate by capability flag.** If the task hits an external service, wrap in `is_enabled('telegram' | 'mail' | …)` so dev branches don't spam.

5. **Verify.** Start dev (`uvicorn backend.api.app:app`), watch `.log/api_log_file` for the task's INFO/DEBUG logs, kill, restart, confirm it picks up cleanly.

 Do not use `time.sleep`. Always `await asyncio.sleep(...)`. Sync sleep blocks the entire event loop.

---

34. Recipe: add a new agent action

Scenario: you want `square_off_underlying` so an agent can close every position on a given underlying.

Steps

1. **Add the handler** in `backend/api/algo/actions.py`. Mirror the shape of `_action_close_position`:

```
async def _action_square_off_underlying(
    action: AgentAction,
    context: dict[str, Any],
) -> ActionResult:
    params = action.params or {}
    underlying = params.get("underlying")
    if not underlying:
        return ActionResult(ok=False, reason="missing underlying")
    ...
```

2. **Register it** in the `_ACTION_HANDLERS` map at the bottom of `actions.py`:

```
_ACTION_HANDLERS["square_off_underlying"] = _action_square_off_underlying
```

3. **Add the grammar token** in `backend/api/algo/grammar.py::_SYSTEM_TOKENS`:

```
{
    "grammar_kind": "action",
    "token_kind": "action_type",
    "token": "square_off_underlying",
    "value_type": "enum",
    "resolver": "backend.api.algo.actions._action_square_off_underlying",
    "params_schema": {
        "required": ["underlying"],
        "properties": {"underlying": {"type": "string"}},
    },
    "is_system": True,
    "is_active": True,
}
```

4. **Mode resolution.** Honor `_resolve_mode()` — never call broker directly. Use `get_broker(account)` and respect the row's mode.
5. **Verify.** Create a test agent in dev via `/admin/tokens` + `/automation`, fire-in-simulator, confirm event row + broker call.

---

35. Recipe: add a new template field (worked example)

Scenario: add `tp_breakeven_lock: bool` — when true, after TP1 fires, modify SL to entry price (free trade).

Steps

1. **Backend column** in `backend/api/models.py::OrderTemplate`:

```
tp_breakeven_lock: Mapped[bool | None] = mapped_column(Boolean, nullable=True)
```

2. **Idempotent ALTER** in `backend/api/database.py::init_db`:

```
await conn.execute(text(
    "ALTER TABLE order_templates ADD COLUMN IF NOT EXISTS tp_breakeven_lock BOOLEAN"
))
```

3. **Schemas** in `backend/api/schemas.py`:
   - `OrderTemplate` (response): `tp_breakeven_lock: bool | None = None`
   - `OrderTemplateCreate`, `OrderTemplatePatch`: same field
   - `TicketOrderRequest`: optional `tp_breakeven_lock_override: bool | None = None` if you want per-submit override
   - `BasketLeg`: same per-leg
4. **Override JSON build** in `backend/api/routes/orders.py::_build_overrides_json` (search for the function):

```
if data.tp_breakeven_lock_override is not None:
    result["tp_breakeven_lock"] = data.tp_breakeven_lock_override
```

5. **Plan resolution** in `backend/api/algo/template_attach.py::resolve_template_plan`:

```
breakeven_lock = _pick("tp_breakeven_lock", template, overrides)
```

   Then emit the appropriate behavior — for breakeven lock, you'd persist it into `attached_gtts_json` and watch for TP1 fill events.

6. **Seeded defaults** in `backend/api/algo/templates_seed.py::SYSTEM_TEMPLATES` — add to whichever defaults should ship with it ON.
7. **Frontend management UI** at `frontend/src/routes/(algo)/automation/templates/+page.svelte`:
   - Add a toggle to the create/edit form
   - Surface it in the listing
8. **Frontend override input** in `frontend/src/lib/SymbolPanel.svelte`:
   - Add a `_sharedTpBreakevenLockOverride = $state(null)` shell-level
   - Render in the Template row alongside the existing override inputs
   - Reset on template change (search the `$effect` block that watches `_sharedTemplateId`)
   - Pass through to OrderTicket and per-leg basket logic
9. **Preview chip.** Backend handles the math; frontend just renders. The preview will automatically pick up the new override because `previewTicketTemplate` passes through the full overrides dict.
10. **Update this doc.** Add a row to §10 (4-default matrix) if any default ships with it, and to §11 (merge order) if your field has unusual merge semantics.
11. **Verify.** `svelte-check`, `pytest`, e2e on `dev.ramboq.com`: create a template with the field, place a paper order, watch the chain of events in `.log/api_log_file`.

---

36. Recipe: add a new broker capability flag

Scenario: you want to track whether each broker supports iceberg orders (currently not in the matrix).

Steps

1. **Add the field** to `backend/brokers/capabilities.py::BrokerCapabilities`:

```
@dataclass(frozen=True)
class BrokerCapabilities:
    ...                         # existing fields
    iceberg_order: bool         # No default — force explicit setting per broker
```

   Note the discipline: real fields don't get defaults (audit B-5 lesson). Every broker constant must set every field.

2. **Set per-broker explicitly** in **all four** constants — KITE/DHAN/GROWW plus the `UNKNOWN_CAPS` fallback at the bottom of `capabilities.py`. Missing UNKNOWN_CAPS will crash the unknown-broker code path at runtime:

```
KITE_CAPS    = BrokerCapabilities(..., iceberg_order=True)
DHAN_CAPS    = BrokerCapabilities(..., iceberg_order=False)
GROWW_CAPS   = BrokerCapabilities(..., iceberg_order=False)
UNKNOWN_CAPS = BrokerCapabilities(..., iceberg_order=False)  # conservative default
```

3. **Capability registry** in `capabilities.py::CAPS_BY_BROKER_ID` already routes by `broker_id` — no change needed.
4. **Frontend warning helper** in `frontend/src/lib/data/brokerCapWarnings.js`:
   - Update the warning-aggregation logic to surface a warning when a template asks for an iceberg leg against a broker where `!caps.iceberg_order`.
5. **Consumer code** queries via `get_broker(account).capabilities.iceberg_order` or the HTTP endpoint `/api/admin/brokers/{account}/capabilities`.
6. **Verify.** Inspect `/admin/brokers` page; the new cap should surface in the row.

---

37. Recipe: add a new page

Scenario: new admin page at `/admin/funds-history`.

Steps

1. **Create the route file** `frontend/src/routes/(algo)/admin/funds-history/+page.svelte`. Copy structure from `/admin/brokers/+page.svelte` — it's the simplest admin page template.
2. **Page header.** Use the canonical pattern (see "Page-header rule" in CLAUDE.md or §17 of this doc):

```
<div class="page-header">
  <span class="algo-title-group">
    <h1 class="page-title-chip">Funds History</h1>
  </span>
  <span class="algo-ts">{$nowStamp}</span>
  <span class="ml-auto"></span>
  <span class="page-header-actions">
    <RefreshButton onClick={load} loading={_loading} label="Refresh" />
    <PageHeaderActions />
  </span>
</div>
```

3. **Navbar entry** in `frontend/src/routes/(algo)/+layout.svelte` `navItems`. Pick a group (`monitor` | `analyze` | `modes` | `build` | `config`). Set `adminOnly: true` if it should hide in demo mode.
4. **Data loading.** Always:
   - Use `$effect` for mount + cleanup (not legacy onMount)
   - Use `marketAwareInterval` from `$lib/stores` for polling (not raw setInterval)
   - Read via `$lib/api.js` wrappers
5. **Verify.** Visit the route, check navbar entry, confirm Refresh works, watch console for errors. Playwright spec if non-trivial.

---

38. Recipe: add a setting

Scenario: add `chase.max_consecutive_errors` so operator can tune the abort threshold (currently hardcoded `_MAX_CHASE_ERRORS=5`).

Steps

1. **Add to `SEEDS`** in `backend/shared/helpers/settings.py`:

```
("chase", "chase.max_consecutive_errors", "int", 5,
 "Abort a chase after this many consecutive API failures", None, "errors"),
```

2. **Read it** wherever the constant was used. Replace `_MAX_CHASE_ERRORS` with `get_int('chase.max_consecutive_errors', 5)`. The cache invalidates on every PATCH; reads are O(1).
3. **Verify.** Visit `/admin/settings` → confirm new row in the Chase bucket. Edit it, watch the change take effect on next chase iteration without restart.

→ **TECH** — the seeder preserves operator overrides on deploy; only the description / schema / default_value refresh. So bumping the default in code only affects fresh installs.

---

39. Recipe: change an existing default template

Scenario: operator wants `default-long-option` to have SL −40% instead of no SL.

Steps

1. **Edit `SYSTEM_TEMPLATES`** in `backend/api/algo/templates_seed.py`:

```
{
    "name": "default-long-option",
    ...
    "sl_pct": 40,  # was None
    "sl_type": "LIMIT",
}
```

2. **Re-seeder behavior.** On startup, `seed_templates` (in `templates_seed.py`) rebuilds system templates by `name` — operator's edits to custom templates are preserved, but system templates are overwritten. **The operator's pulls of `default-long-option` will get the new SL on next deploy.**
3. **If operator has saved-instance edits** (i.e. clicked Edit on a system template and saved), those land in a separate row keyed by user_id. They survive system re-seed. To force-refresh, the operator deletes their saved copy.
4. **Verify.** Restart dev, hit `/automation/templates`, confirm SL value is updated. Place a test order — fill should trigger SL GTT placement.

---

40. Recipe: wire a new notification channel

Scenario: add Slack as a notify channel alongside Telegram + email.

Steps

1. **Add config keys** in `backend/config/secrets.yaml` (manually on server, gitignored):

```
slack_webhook_url: "https://hooks.slack.com/services/..."
```

2. **Add capability flag** in `backend/config/backend_config.yaml::cap_in_dev`:

```
slack: True
```

`is_enabled('slack')` returns `True` on main always, else respects this flag.

3. **Add the helper** in `backend/shared/helpers/alert_utils.py`:

```
def _send_slack(message: str) -> None:
    if not is_enabled('slack'):
        return
    webhook = secrets.get('slack_webhook_url')
    if not webhook:
        return
    requests.post(webhook, json={"text": message}, timeout=5)
```

4. **Add the grammar token** for the notify channel:

```
# backend/api/algo/grammar.py::_SYSTEM_TOKENS
{
    "grammar_kind": "notify",
    "token_kind": "channel",
    "token": "slack",
    ...
}
```

5. **Wire it in `_dispatch`** in `alert_utils.py` — for each notify event, check `channel == 'slack'` and call `_send_slack`.
6. **UI checkbox** in agent editor (`frontend/src/routes/(algo)/automation/+page.svelte`) — add Slack to the events grid.
7. **Verify.** Create a test agent with Slack notify, fire in simulator, confirm message lands.

---

41. Recipe: ship a fix to dev + main

Scenario: you've finished a small fix on `dev` branch and want to deploy to prod.

Steps

1. **Run pre-flight checks locally.**
   - `cd backend && pytest` (or scoped to affected files)
   - `cd frontend && npm run check` (svelte-check)
   - `cd frontend && npx playwright test --workers=1` if frontend-touching
2. **Commit on dev.** Use the conventional format:

```
<Sprint/scope>: <one-line summary>

<optional body explaining why>

Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>
```

3. **Push dev.** `git push origin dev`. Webhook triggers dev deploy automatically. Watch logs:

```
ssh ramboq "tail -f /opt/ramboq_dev/.log/api_log_file"
```

4. **Verify on `dev.ramboq.com`.** Manual smoke test of the changed feature.
5. **Merge to main.**

```
git checkout main
git merge --ff-only dev
git push origin main
git checkout dev  # back to dev for next change
```

6. **Webhook deploys prod automatically.** Watch `/opt/ramboq/.log/api_log_file`.
7. **Telegram deploy ping** lands in `RamboQuant Alerts` group — confirm.

 Don't squash-merge. We keep linear history via `--ff-only`. Don't tag releases; we deploy on every push.

---

42. Cross-cutting checklist before every commit

Run mentally before every commit. Skipping any of these has burned us before:

- ☐ **Demo mode honored?** Read paths mask accounts; write paths return 403 or downgrade to paper (§22).
- ☐ **Idempotency on side-effects?** Anything that places orders/GTTs needs a guard (§3.2).
- ☐ **Mode resolution?** Order code reads `row.mode` instead of branching by `if live` (§3.4).
- ☐ **Logger discipline?** No `print()`; no logger calls in hot loops (§25).
- ☐ **Hot-loop sleep?** `asyncio.sleep`, never `time.sleep` (§4.3).
- ☐ **TODO/FIXME?** None in code paths; finish or extract.
- ☐ **Old tests still pass?** `pytest` on the closest test file.
- ☐ **svelte-check clean?** No new errors introduced.
- ☐ **CLAUDE.md updated?** If a fact in CLAUDE.md changes, fix it here.
- ☐ **DESIGN_GUIDE.md updated?** This file. Especially recipe sections + line-anchored grep targets if functions moved.
- ☐ **Sprint/audit fix labeled?** Commit body mentions the gap ID or sprint letter so `git log --grep` finds it.
- ☐ **Co-author trailer?** Co-Authored-By line at the end of the commit body.

If you can't tick every box, hold the commit. The cost of pausing is low; the cost of regression is high.

Where to learn more

If you want to learn…	Read
How the operator uses the platform end-to-end	`USER_GUIDE.md`
Day-to-day operations + troubleshooting	`ADMIN_GUIDE.md`
Agent authoring + testing	`AGENTS_GUIDE.md`
Simulator scenarios + Run-in-Simulator	`SIMULATOR_GUIDE.md`
Lab + MCP-driven research workflow	`LAB_MCP_GUIDE.md`

If you want to learn…	Read
Operator's-eye-view of every page	`CLAUDE.md` (large, but indexed)
Architecture + change recipes	**This document**

Glossary

Curated index of the terms that appear repeatedly in this guide. Alphabetized so you can jump-lookup without scrolling the TOC.

Term	Meaning
Action	Side-effect an agent performs when its condition fires — place order, close position, alert. See §34.
AgentEngine	Declarative rule runner. Reads `agents` table + built-in `BUILTIN_AGENTS`, evaluates conditions each cycle, dispatches actions. §22.5, §34.
Alert	Runtime event produced when an agent condition fires. Persisted to `agent_events`; may or may not have a Notify or Action. Distinct from **Notify**.
Basket order	Multi-account / multi-leg order dispatched atomically. `POST /api/orders/basket` groups by account and fans out via `asyncio.gather`. §6, §22.10.
Broker abstraction	Uniform Python interface (Kite / Dhan / Groww adapters) so route handlers stay broker-agnostic. `backend/brokers/adapters/`. §14.
Chase loop	Adaptive limit-order engine that re-quotes toward the touch until filled or capped. Spread-aware. §7, §12.
cap_in_dev	Nested dict of capability flags in `backend_config.yaml`. All True on `main`, per-branch on dev. Gated via `is_enabled('<cap>')`.
close_settled	Second phase of the snapshot lifecycle — fires 15 min after `<exch>:close`. UPSERTs broker's weighted-avg-last-30-min close price.
conn_service	Standalone Litestar app on `/tmp/ramboq_conn.sock` that owns broker sessions (Kite WebSocket, Dhan/Groww tokens). Restart independent of the main API.
Demo mode	Signed-out + prod branch — read-only + PII-masked. Write paths return 403 or degrade to paper. §22.
Firm NAV	`cash_sod + option_premium + Σ position.unrealised + Σ holdings.cur_val`. Canonical formula (v4) in `backend/api/algo/nav.py:compute_firm_nav`.
GTT	Good-Till-Triggered order — broker-side conditional order. Kite native; Dhan OCO leg; Groww emulated. §9.
@for_all_accounts	Decorator that fans out a broker call across every account and concatenates the results. `pd.concat(..., ignore_index=True)` at call site.
Idempotency guard	Column like `attached_gtts_json IS NULL` that ensures a side-effect fires at most once even under postback retries. §3.2.
KiteTicker	Persistent Kite WebSocket. One per conn_service process. Ticks land in `/dev/shm/ramboq_ticks` mmap; main API reads via `MmapTickReader`.
LP unit	Limited-Partner accounting unit. `units_held × nav_per_unit = slice`. §22.6.
MarketPulse	Two-side ag-Grid page (`/pulse`) — pinned watchlists + movers left, positions + holdings right. Canonical operator surface.
MCP	Model Context Protocol — Claude-Code integration exposing 25 platform tools (17 read-only + 2 persist + 6 write-gated).
Notify	Delivery channel wrapping an Alert — telegram / email / websocket / log. Vocabulary chain: Agent □ Alert □ Notify □ Action.

Term	Meaning
OHLCV	Open-High-Low-Close-Volume daily bars. Cached in `ohlcv_store` (3-tier: LRU → PostgreSQL → broker). 5-year retention.
Paper mode	Live-quote execution against `PaperTradeEngine` — no broker orders. Mode 2 of the confidence ladder (sim → paper → shadow → live).
PBKDF2	Password hash algorithm (SHA-256, 210k iters). JWT signing separate — HS256.
Preflight	Fat-finger + lot-multiple guards before order placement (G1, G2). Parallelized via `asyncio.gather`. §22.10.
Proxy hedge	Cross-reference between holdings and option roots. β-regressed. `hedge_proxies` table.
Refresh cycle mode	Persistence override — `off` / `soft` / `hard`. Bypasses cache tiers when defect-recovering. Runtime-only, resets on restart.
Shadow mode	Log-only mode — validates payload against real broker but does not execute. Mode 5, prod-only.
snapshot_extras	Payload block in `daily_book.payload_json` carrying open/high/low/close_settled/day_change_val/… for closed-hours reads.
Snapshot lifecycle	Per-exchange event sequence — `open` → `close` (first-cut snapshot) → `close_settled` (broker's settled close).
SSOT	Single Source Of Truth. See `baseDayPnlForPosition` (Day P&L), `compute_firm_nav` (NAV), `resolve_current_price` (LTP resolver).
Template attach	Post-fill automation — attaches GTT exits and take-profit legs to a filled parent. Idempotent via `parent_order_id`. §9.
Ticker mmap	`/dev/shm/ramboq_ticks` — fixed 4096-slot shared-memory buffer, version-word atomic, lock-free reads. Main API tails it via `MmapTickReader`.
Trail stop	Stop-loss that ratchets toward the touch on favorable moves. Broker-side (Kite trail_gtt) or emulated. §13.
Virtual root	MCX/CDS synthetic symbol (e.g. `CRUDEOIL` = front-month, `CRUDEOIL_NEXT` = back-month). Resolver: `symbol_resolver.py`.

Alphabetical section index

Quick-jump index by first significant word — useful when you remember a name but not the section number.

Section	§
Architecture overview	§1
Audit log — forensic trail	§22.7
Background task topology	§20
Broker abstraction	§14
Broker abstraction — implementation detail	§14.5
Broker gotchas	§16
Chart indicator system	§22.15
Chase loop invariants	§12
Chase loop lifecycle	§7
Concurrency model	§4
Core architectural principles	§3
Cross-cutting checklist before every commit	§42

Section	§
Data layer — implementation detail	§4.5
Data refresh — PositionStrip + Dashboard	§21
Database schema overview	§4.6
Demo mode	§22
Deployment notes	§26
History — orders / trades / funds	§22.9
Investor portal — token-as-credential	§22.5
Investor portal — units-based NAV math	§22.6
Logging discipline	§25
Market-status probe	§22.14
Metrics + performance tracking	§4.9
Navbar audit — rename + resequence	§22.11
Operator's mental model	§30
Order placement — basket (Chain tab)	§6
Order placement — single ticket	§5
Order placement latency	§22.10
Postback fan-out — book_changed bus	§22.8
Reading order for a new developer	§28
Recipe — add a background task	§33
Recipe — add a broker capability flag	§36
Recipe — add a column to a table	§32
Recipe — add a new agent action	§34
Recipe — add a new page	§37
Recipe — add a new route	§31
Recipe — add a new template field	§35
Recipe — add a setting	§38
Recipe — change a default template	§39
Recipe — ship a fix to dev + main	§41
Recipe — wire a notification channel	§40
Retention policies	§4.8
Sprint history + audit fixes	§27
Table relationships	§4.7

Section	§
Tech stack — at a glance	§2
Template attach pipeline	§9
Template matrix (4 defaults)	§10
Template override merge	§11
Testing philosophy	§24
The order/chase/template tripod	§8
The preview pipeline	§19
Trail-stop subsystem	§13
UX audit slice D	§22.13
When in doubt	§29
#audit workflow + postback scaffold	§22.12